

14 Binary I/O



Motivations

Data stored in a text file is represented in human-readable form. Data stored in a binary file is represented in binary form. You cannot read binary files. They are designed to be read by programs.

For example, Java source programs are stored in text files and can be read by a text editor, but Java classes are stored in binary files and are read by the JVM. The advantage of binary files is that they are more efficient to process than text files.



Objectives

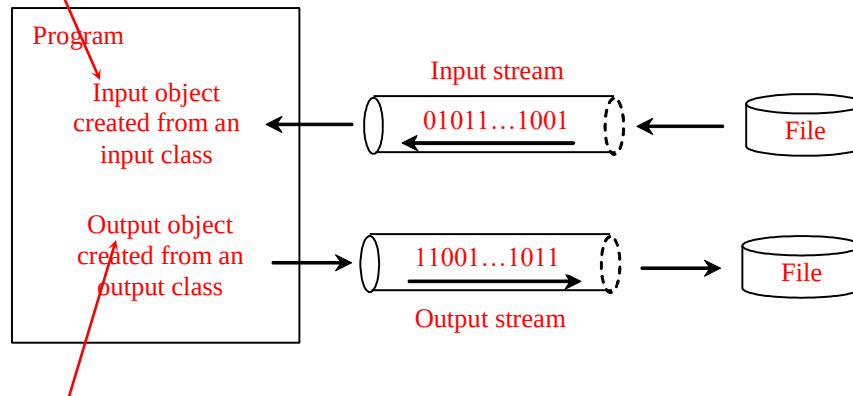
- ❑ To discover how I/O is processed in Java (§17.2).
- ❑ To distinguish between text I/O and binary I/O (§17.3).
- ❑ To read and write bytes using `FileInputStream` and `FileOutputStream` (§17.4.1).
- ❑ To read and write primitive values and strings using `DataInputStream/DataOutputStream` (§17.4.3).
- ❑ To store and restore objects using `ObjectOutputStream` and `ObjectInputStream`, and to understand how objects are serialized and what kind of objects can be serialized (§17.6).
- ❑ To implement the `Serializable` interface to make objects serializable (§17.6.1).
- ❑ To serialize arrays (§17.6.2).
- ❑ To read and write the same file using the `RandomAccessFile` class (§17.7).



How is I/O Handled in Java?

A File object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file. In order to perform I/O, you need to create objects using appropriate Java I/O classes.

```
Scanner input = new Scanner(new File("temp.txt"));  
System.out.println(input.nextLine());
```



```
PrintWriter output = new PrintWriter("temp.txt");  
output.println("Java 101");  
output.close();
```



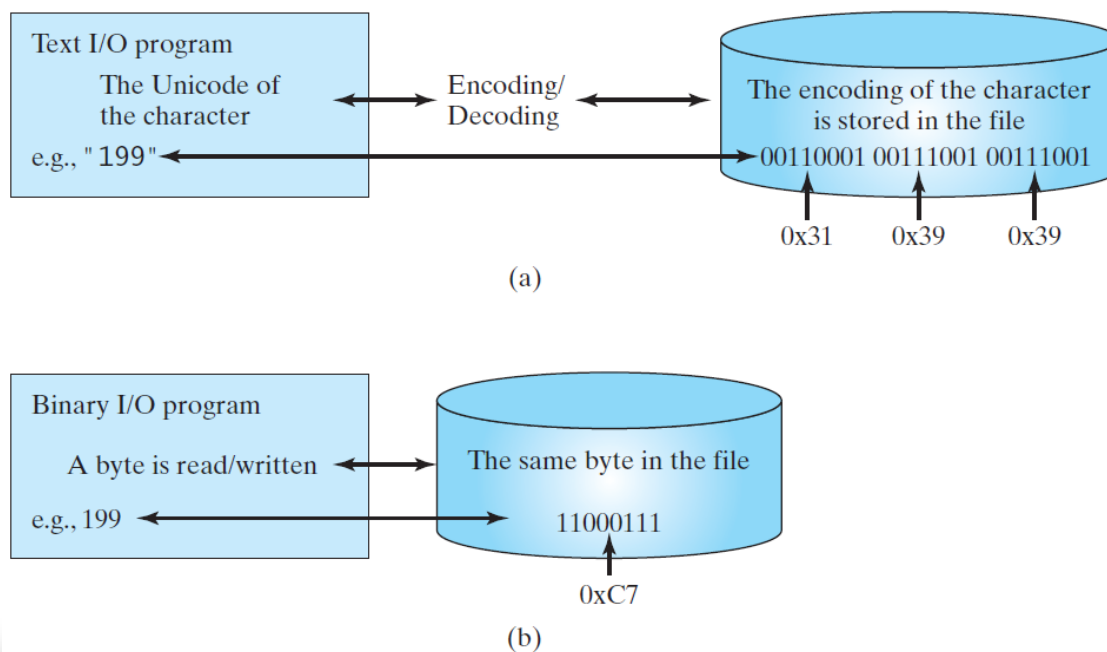
Text File vs. Binary File

- ❑ Data stored in a text file are represented in human-readable form. Data stored in a binary file are represented in binary form. You cannot read binary files. Binary files are designed to be read by programs. For example, the Java source programs are stored in text files and can be read by a text editor, but the Java classes are stored in binary files and are read by the JVM. The advantage of binary files is that they are more efficient to process than text files.
- ❑ Although it is not technically precise and correct, you can imagine that a text file consists of a sequence of characters and a binary file consists of a sequence of bits. For example, the decimal integer 199 is stored as the sequence of three characters: '1', '9', '9' in a text file and the same integer is stored as a byte-type value C7 in a binary file, because decimal 199 equals to hex C7.

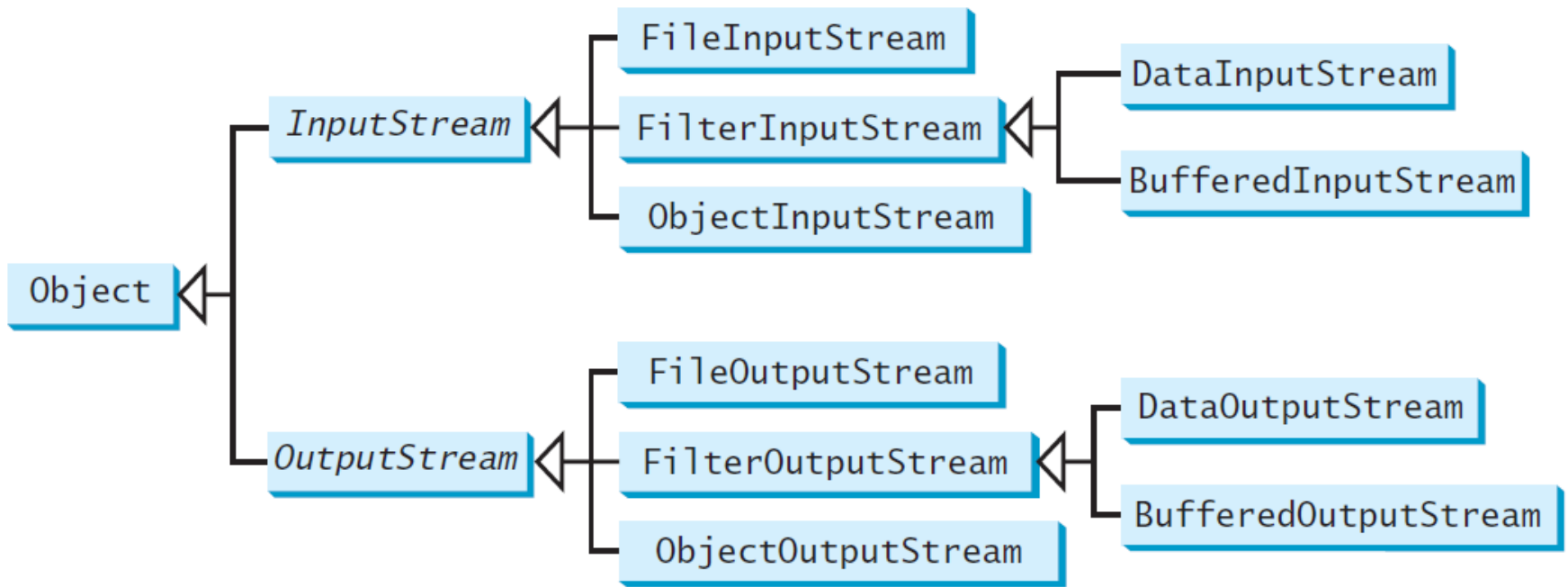
Binary I/O

Text I/O requires encoding and decoding. The JVM converts a Unicode to a file specific encoding when writing a character and converts a file specific encoding to a Unicode when reading a character.

Binary I/O does not require conversions. When you write a byte to a file, the original byte is copied into the file. When you read a byte from a file, the exact byte in the file is returned.



Binary I/O Classes



InputStream

The value returned is a byte as an int type.

java.io.InputStream

+read(): int

Reads the next byte of data from the input stream. The value byte is returned as an int value in the range 0 to 255. If no byte is available because the end of the stream has been reached, the value -1 is returned.

+read(b: byte[]): int

Reads up to b.length bytes into array b from the input stream and returns the actual number of bytes read. Returns -1 at the end of the stream.

+read(b: byte[], off: int, len: int): int

Reads bytes from the input stream and stores into b[off], b[off+1], ..., b[off+len-1]. The actual number of bytes read is returned. Returns -1 at the end of the stream.

+available(): int

Returns the number of bytes that can be read from the input stream.

+close(): void

Closes this input stream and releases any system resources associated with the stream.

+skip(n: long): long

Skips over and discards n bytes of data from this input stream. The actual number of bytes skipped is returned.

+markSupported(): boolean

Tests if this input stream supports the mark and reset methods.

+mark(readlimit: int): void

Marks the current position in this input stream.

+reset(): void

Repositions this stream to the position at the time the mark method was last called on this input stream.

OutputStream

The value is a byte as an int type.

java.io.OutputStream

+write(int *b*): void

Writes the specified byte to this output stream. The parameter *b* is an int value. (byte)*b* is written to the output stream.

+write(*b*: byte[]): void

Writes all the bytes in array *b* to the output stream.

+write(*b*: byte[], *off*: int, *len*: int): void

Writes *b*[*off*], *b*[*off*+1], ..., *b*[*off*+*len*-1] into the output stream.

+close(): void

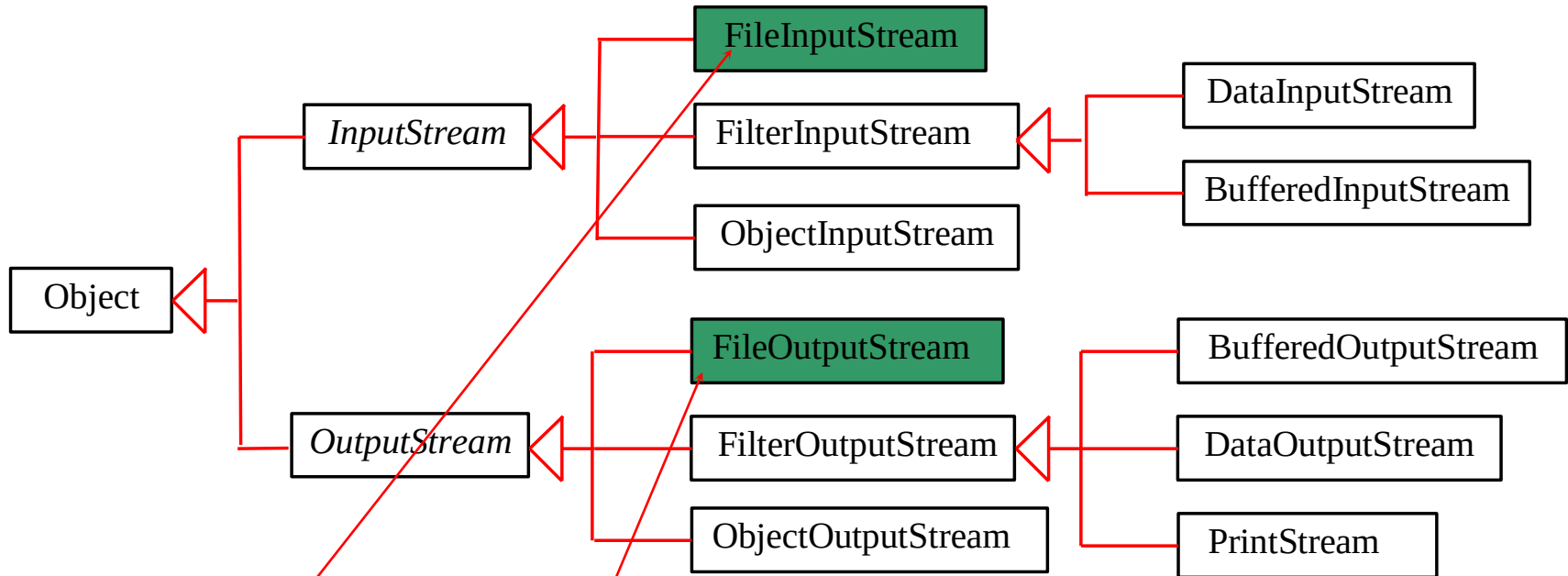
Closes this output stream and releases any system resources associated with the stream.

+flush(): void

Flushes this output stream and forces any buffered output bytes to be written out.



FileInputStream/FileOutputStream



FileInputStream/FileOutputStream associates a binary input/output stream with an external file. **All the methods** in FileInputStream/FileOuptputStream are inherited from its superclasses.



FileInputStream

To construct a `FileInputStream`, use the following constructors:

```
public FileInputStream(String filename)
```

```
public FileInputStream(File file)
```

A `java.io.FileNotFoundException` would occur if you attempt to create a `FileInputStream` with a nonexistent file.



FileOutputStream

To construct a `FileOutputStream`, use the following constructors:

```
public FileOutputStream(String filename)  
public FileOutputStream(File file)  
public FileOutputStream(String filename, boolean append)  
public FileOutputStream(File file, boolean append)
```

If the file does not exist, a new file would be created. If the file already exists, the first two constructors would delete the current contents in the file. To retain the current content and append new data into the file, use the last two constructors by passing true to the append parameter.



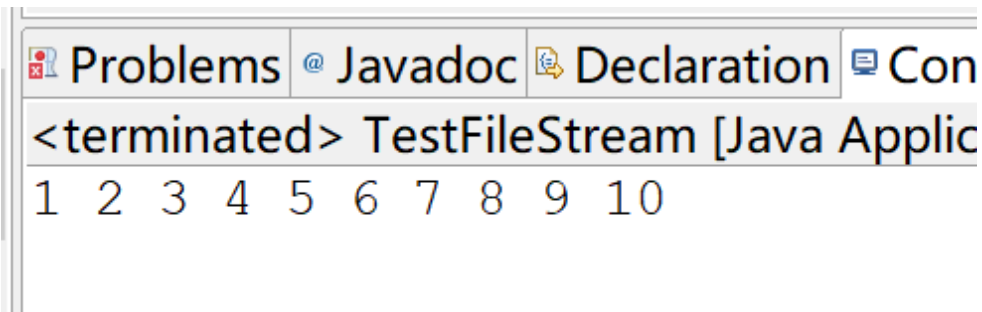
TestFileStream

Run

```

1 import java.io.*;
2
3 public class TestFileStream {
4     public static void main(String[] args) throws IOException {
5         try (
6             // Create an output stream to the file
7             FileOutputStream output = new FileOutputStream("temp.dat");
8         ) {
9             // Output values to the file
10            for (int i = 1; i <= 10; i++)
11                output.write(i);
12        }
13
14        try (
15            // Create an input stream for the file
16            FileInputStream input = new FileInputStream("temp.dat");
17        ) {
18            // Read values from the file
19            int value;
20            while ((value = input.read()) != -1)
21                System.out.print(value + " ");
22        }
23    }
24 }

```





temp.dat



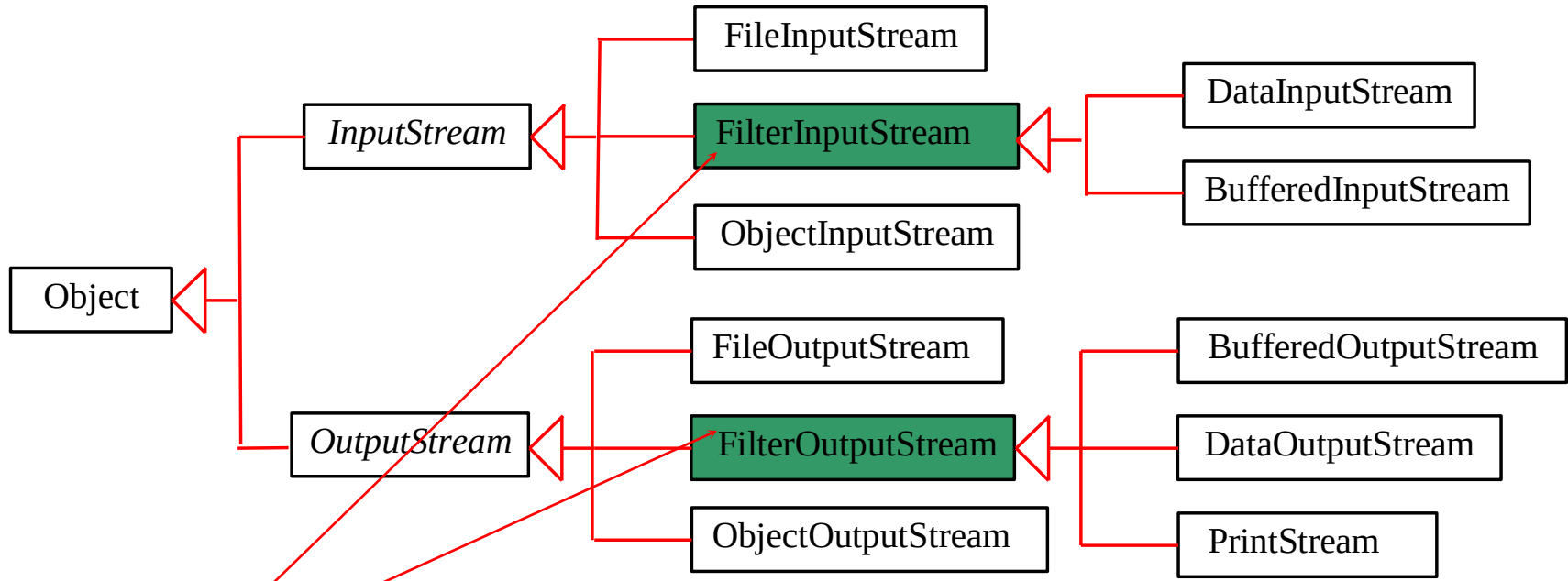
1

SOHSTXETXEOTENOACKBELBS

2



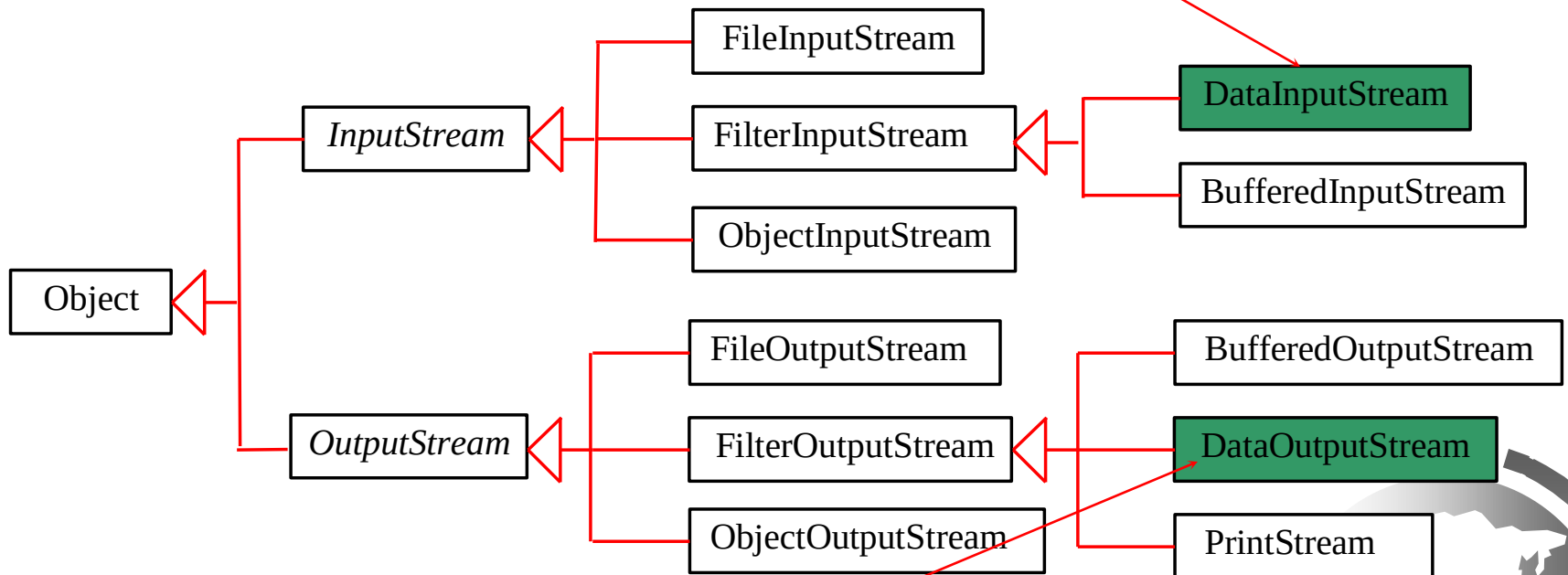
FilterInputStream/FilterOutputStream



Filter streams are streams that filter bytes for some purpose. **The basic byte input stream provides a read method that can only be used for reading bytes.** If you want to read integers, doubles, or strings, you **need a filter class to wrap the byte input stream.** Using a filter class enables you to read integers, doubles, and strings instead of bytes and characters. FilterInputStream and FilterOutputStream are the base classes for filtering data. When you need to process primitive numeric types, use DataInputStream and DataOutputStream to filter bytes.

DataInputStream/DataOutputStream

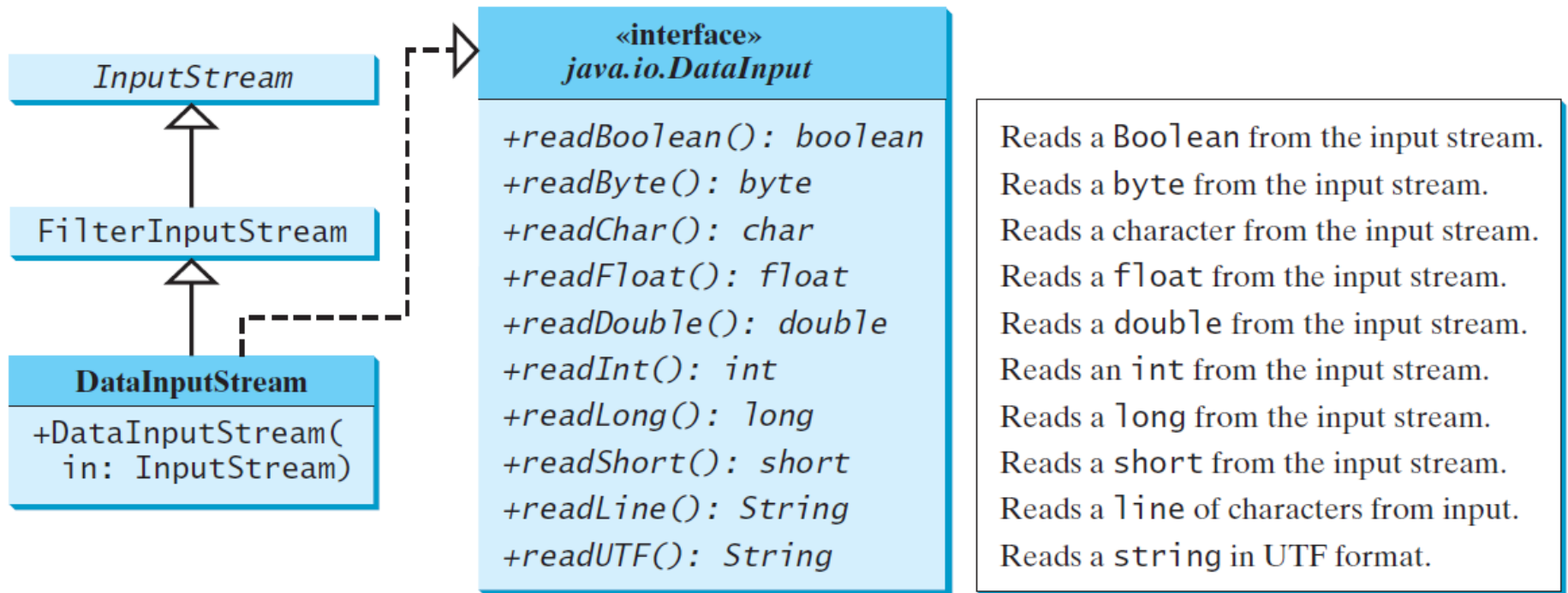
DataInputStream reads bytes from the stream and converts them into appropriate primitive type values or strings.



DataOutputStream converts primitive type values or strings into bytes and output the bytes to the stream.

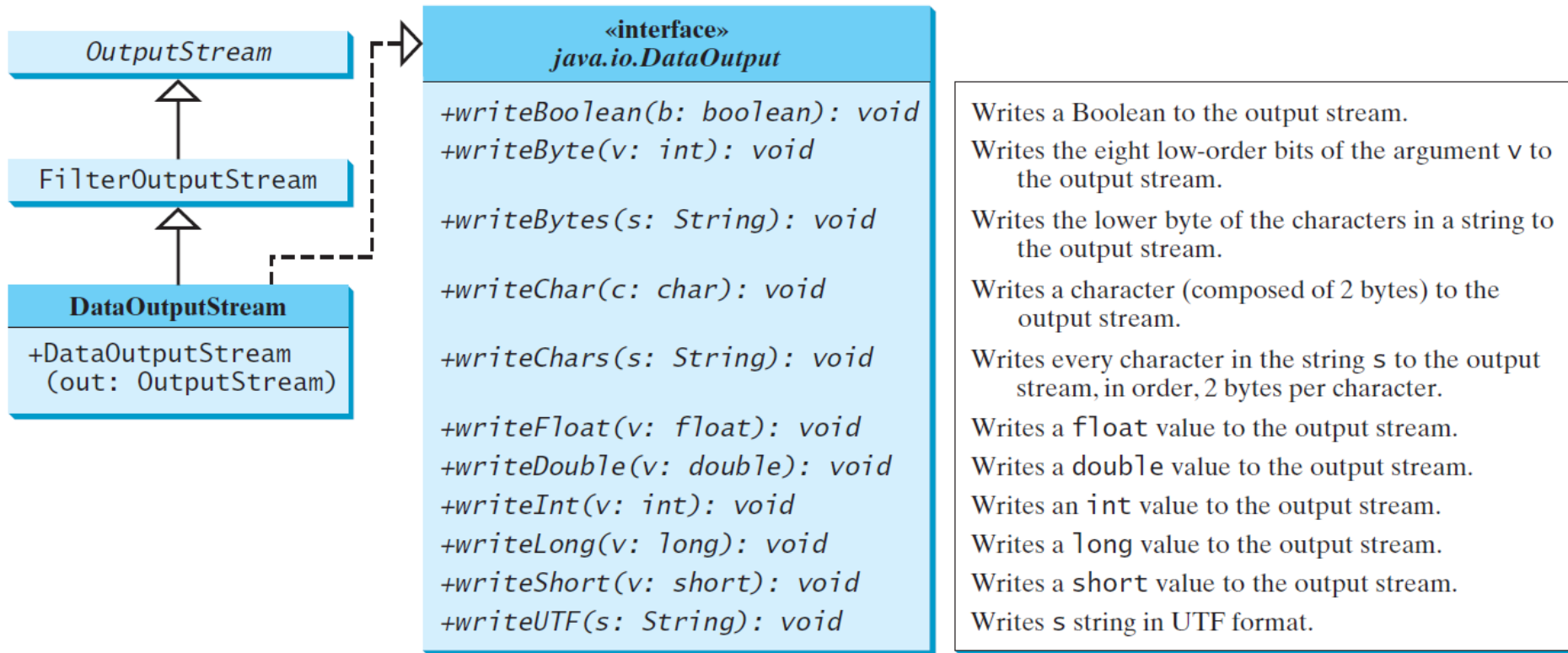
DataInputStream

`DataInputStream` extends `FilterInputStream` and implements the `DataInput` interface.



DataOutputStream

DataOutputStream extends FilterOutputStream and implements the DataOutput interface.



Characters and Strings in Binary I/O

A Unicode consists of two bytes. The `writeChar(char c)` method writes the Unicode of character `c` to the output. The `writeChars(String s)` method writes the Unicode for each character in the string `s` to the output.

Why UTF-8? What is UTF-8?

UTF-8 is a coding scheme that allows systems to operate with both ASCII and Unicode efficiently. **Most operating systems use ASCII. Java uses Unicode.** The ASCII character set is a subset of the Unicode character set. Since most applications need only the ASCII character set, it is a waste to represent an 8-bit ASCII character as a 16-bit Unicode character. **The UTF-8 is an alternative scheme that stores a character using 1, 2, or 3 bytes.** ASCII values (less than 0x7F) are coded in one byte. Unicode values less than 0x7FF are coded in two bytes. Other Unicode values are coded in three bytes.

字符集	定义	简介
ASCII	美国信息交换标准码 (American Standard Code for Information Interchange)	7位编码，二进制取值范围0000000~1111111 (共128个字符) - 存储时依然是以8位存储 - 包含控制字符 和 图形字符 - 重要字符：0 - 48、A - 65、a - 97 (大小写字母相差32)
GB类	先后出现GB2312、GBK、GB18030等字符集	- 双字节定长编码 - 汉字的字符编码集，从GBK开始支持繁体字
Big5	五大码	最早是台湾省/港澳使用的繁体字编码集，与GB类不兼容
ANSI	美国国家标准学会 (American National Standards Institute)	- ANSI并不是某种特定的字符编码，而是在不同的系统中，ANSI表示不同的字符编码 - 在英文系统中，默认为ASCII - 在简体中文系统中，默认为GB类 - 在繁体中文系统中，默认为Big-5 - 命令行使用chcp命令，可显示或者设置当前系统ANSI的编码方式 (936 简体中文 950 繁体中文)
USC	(Universal Coded Character Set) 1991年10月，两个团队决定合并字符集，后来两个字符集合并为一，UCS的码点将与Unicode完全一致	- 国际标准化组织 (ISO) 创建 - UCS-2: 2个字节定长编码，包含65536个编码空间 (可以为全世界最常用的63K字符编码，为了兼容Unicode，0xD800-0xDFFF之间的码位未使用) UCS-2是UTF-16的一个子集 - UCS-4: 4个字节定长编码 编码空间为0x00000000~0xFFFFFFFF (可以编码20多亿个字符)，为了兼容Unicode标准，只不过其编码空间被限定在了0~0x10FFFF之间 UTF-32是UCS-4的一个子集
Unicode	统一化字符编码标准 (The Unicode Standard)，或称统一码	- 统一码联盟创建 - 统一包含了世界上所有字符，每个符号都有唯一的一个码点 - 三种编码方式：UTF-8、UTF-16、UTF-32

字符编码问题

编码分类

- ASCII 码
 - 1 个字节，最多127个字符
 - 最高位为0
- Unicode
 - 2 个字节，复杂的用4个字节
- UTF-8
 - 可变长，1-6字节
 - 英文1字节，汉字3字节，复杂的4-6字节
- gb2312
 - 汉字特有的编码
 - 2 字节

计算机中实际表示

- 在计算内存中，统一用 Unicode
- 保存到磁盘或者传输的时候就用 UTF-8

Characters and Code

String底层使用char数组来存储数据，每个char占用两个字节，每个char表示一个Unicode，String实际存储的是文字的Unicode序列。

```
1 String str = "你好吗";
2 char[] chs = str.toCharArray();
3 for (char ch: chs) {
4     System.out.printf("%04x\n", (int)ch);
5 }
6 /*
7 输出:
8 4f60
9 597d
10 5417
11 */
```

“你好吗”对应的Unicode编码分别为4f60、597d、5417，和上例中的输出一致。

InputStreamReader和OutputStreamWriter可以用来读写各种编码格式的文本文件，使用示例如下：

```
1 public class Test {
2     void writerText(String text, String charSetName) throws IOException{
3         OutputStreamWriter ow = new OutputStreamWriter(new FileOutputStream("rw.txt"),
4             Charset.forName(charSetName));
5         ow.write(text);
6         ow.close();
7     }
8     void readText(String charSetName) throws IOException {
9         InputStreamReader ir = new InputStreamReader(new FileInputStream("rw.txt"),
10             Charset.forName(charSetName));
11         char[] charBuf = new char[1024];
12         int n = ir.read(charBuf);
13         for (int i=0; i<n; i++){
14             System.out.print(charBuf[i]);
15         }
16         System.out.println();
17     }
18     public static void main(String[] args) throws IOException{
19         Test t = new Test();
20         t.writerText("你好java", "GBK");
21         t.readText("GBK");
22         t.readText("UTF-8");
23     }
24 }
25 /*
```

/*

输出：

你好java

???java

*/

(____) will be the result of running the following code.

```
String s = "AB浙江大学CD";  
byte[] b = s.getBytes();  
System.out.println(b.length+"->" + s.length());
```

- ☐ A. 12->8
- ☐ B. Not sure, depending on the source character set
- ☐ C. 16->8
- ☐ D. 16->10




```

/**
 * Encodes this {@code String} into a sequence of bytes using the
 * platform's default charset, storing the result into a new byte array.
 *
 * <p> The behavior of this method when this string cannot be encoded in
 * the default charset is unspecified. The {@link
 * java.nio.charset.CharsetEncoder} class should be used when more control
 * over the encoding process is required.
 *
 * @return The resultant byte array
 *
 * @since      JDK1.1
 */
public byte[] getBytes() {
    return StringCoding.encode(value, 0, value.length);
}

```



Using DataInputStream/DataOutputStream

Data streams are used as **wrappers** on existing input and output streams to filter data in the original stream. They are created using the following constructors:

```
public DataInputStream(InputStream instream)
public DataOutputStream(OutputStream outstream)
```

The statements given below create data streams. The first statement creates an input stream for file **in.dat**; the second statement creates an output stream for file **out.dat**.

```
DataInputStream infile =
    new DataInputStream(new FileInputStream("in.dat"));
DataOutputStream outfile =
    new DataOutputStream(new FileOutputStream("out.dat"));
```



TestDataStream

Run



```
public class TestDataStream {
    public static void main(String[] args) throws IOException {
        try ( // Create an output stream for file temp.dat
            DataOutputStream output =
                new DataOutputStream(new FileOutputStream("temp.dat"));
        ) {
            // Write student test scores to the file
            output.writeUTF("John");
            output.writeDouble(85.5);
            output.writeUTF("Jim");
            output.writeDouble(185.5);
            output.writeUTF("George");
            output.writeDouble(105.25);
        }

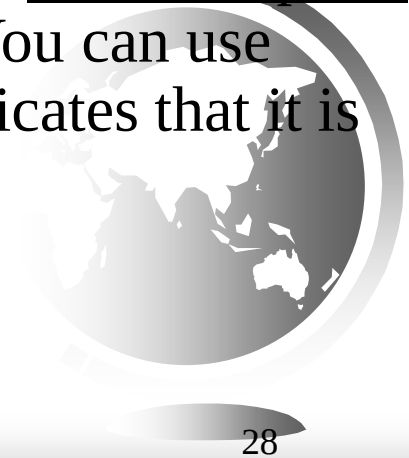
        try ( // Create an input stream for file temp.dat
            DataInputStream input =
                new DataInputStream(new FileInputStream("temp.dat"));
        ) {
            // Read student test scores from the file
            System.out.println(input.readUTF() + " " + input.readDouble());
            System.out.println(input.readUTF() + " " + input.readDouble());
            System.out.println(input.readUTF() + " " + input.readDouble());
        }
    }
}
```

Order and Format

CAUTION: You have to read the data in the same order and same format in which they are stored. For example, since names are written in UTF-8 using writeUTF, you must read names using readUTF.

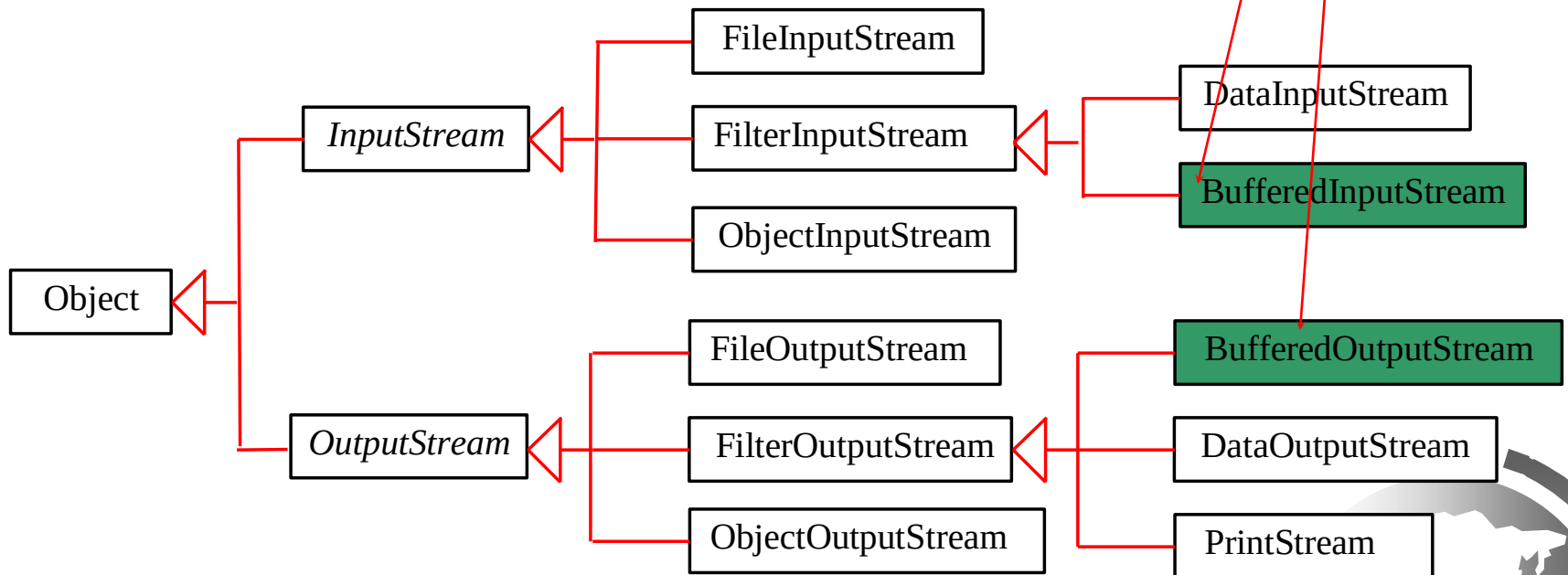
Checking End of File

TIP: If you keep reading data at the end of a stream, an EOFException would occur. So how do you check the end of a file? You can use input.available() to check it. input.available() == 0 indicates that it is the end of a file.



BufferedInputStream/ BufferedOutputStream

Using buffers to speed up I/O



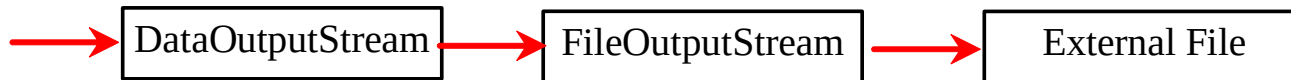
BufferedInputStream/BufferedOutputStream does not contain new methods. All the methods BufferedInputStream/BufferedOutputStream are inherited from the InputStream/OutputStream classes.

Concept of pipe line



int, double, string ...

01000110011 ...



int, double, string ...

01000110011 ...



Constructing BufferedInputStream/BufferedOutputStream

// Create a BufferedInputStream

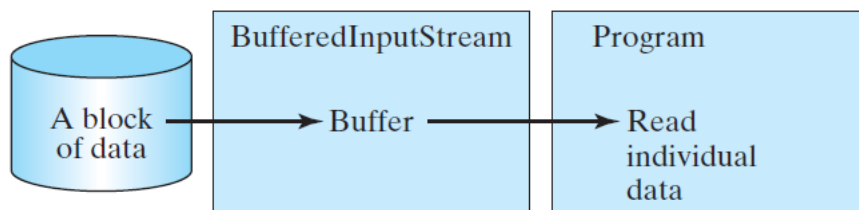
```
public BufferedInputStream(InputStream in)
```

```
public BufferedInputStream(InputStream in, int bufferSize)
```

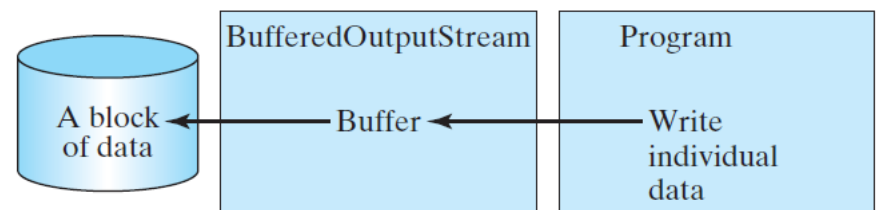
// Create a BufferedOutputStream

```
public BufferedOutputStream(OutputStream out)
```

```
public BufferedOutputStream(OutputStreamr out, int bufferSize)
```



(a)

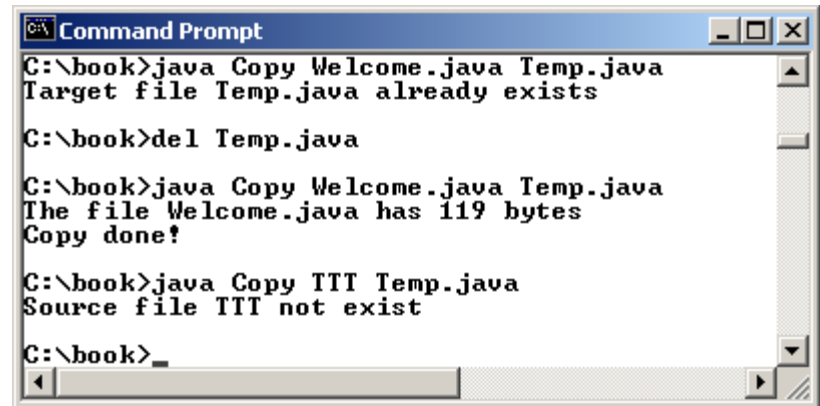


(b)

Case Studies: Copy File

This case study develops a program that copies files. The user needs to provide a source file and a target file as command-line arguments using the following command:

java Copy source target



```
Command Prompt
C:\book>java Copy Welcome.java Temp.java
Target file Temp.java already exists

C:\book>del Temp.java

C:\book>java Copy Welcome.java Temp.java
The file Welcome.java has 119 bytes
Copy done!

C:\book>java Copy TTT Temp.java
Source file TTT not exist

C:\book>
```

The program copies a source file to a target file and displays the number of bytes in the file. If the source does not exist, tell the user the file is not found. If the target file already exists, tell the user the file already exists.



Copy

Run

FilterInputStream 与 装饰者模式

一般有两种方式可以实现给一个类或对象增加行为：

- **继承机制**，使用继承机制是给现有类添加功能的一种有效途径，通过继承一个现有类可以使得子类在拥有自身方法的同时还拥有父类的方法。但是这种方法是静态的，用户不能控制增加行为的方式和时机。
- **关联机制**，即将一个类的对象嵌入另一个对象中，由另一个对象来决定是否调用嵌入对象的行为以便扩展自己的行为，我们称这个嵌入的对象为装饰器 (Decorator)。



FilterInputStream 与 装饰者模式

```
package java.io;
```

```
public class FilterInputStream extends InputStream {
```

```
    /**
```

```
     * The input stream to be filtered.
```

```
     */
```

```
    protected volatile InputStream in;
```

```
    /**
```

```
     * Creates a FilterInputStream
```

```
     * by assigning the argument in
```

```
     * to the field this.in so as
```

```
     * to remember it for later use.
```

```
     *
```

```
     * @param in the underlying input stream, or null if
```

```
     * this instance is to be created without an underlying stream.
```

```
     */
```

```
    protected FilterInputStream(InputStream in) {
```

```
        this.in = in;
```

```
    }
```

```
    public int read() throws IOException {
```

```
        return in.read();
```

```
    }
```

```
    public int read(byte b[]) throws IOException {
```

```
        return read(b, 0, b.length);
```

```
    }
```

```
    public int read(byte b[], int off, int len) throws IOException {
```

```
        return in.read(b, off, len);
```

```
    }
```

```
    public void close() throws IOException {
```

```
        in.close();
```

```
    }
```

FilterInputStream 与 装饰者模式

FilterInputStream子类可以分成两类：

- 1) DataInputStream能以一种与机器无关的方式，直接从地从字节输入流读取JAVA基本类型和String类型的数据。
- 2) 其它的子类使得能够对InputStream进行改进，即在原有的InputStream基础上可以提供了新的功能特性。日常中用的最多的就是BufferedInputStream，使得inputStream具有缓冲的功能。



FilterInputStream 与 装饰者模式

DataInputStream: 之前的InputStream我们只能读取byte，这个类使得我们可以直接从stream中读取int，String等类型。

```
public final int readInt() throws IOException {  
    int ch1 = in.read();  
    int ch2 = in.read();  
    int ch3 = in.read();  
    int ch4 = in.read();  
    if ((ch1 | ch2 | ch3 | ch4) < 0)  
        throw new EOFException();  
    return ((ch1 << 24) + (ch2 << 16) + (ch3 << 8) + (ch4 << 0));  
}
```



```

501     public final String readLine() throws IOException {
502         char buf[] = lineBuffer;
503
504         if (buf == null) {
505             buf = lineBuffer = new char[128];
506         }
507
508         int room = buf.length;
509         int offset = 0;
510         int c;
511
512 loop:    while (true) {
513             switch (c = in.read()) {
514                 case -1:
515                 case '\n':
516                     break loop;
517
518                 case '\r':
519                     int c2 = in.read();
520                     if ((c2 != '\n') && (c2 != -1)) {
521                         if (!(in instanceof PushbackInputStream)) {
522                             this.in = new PushbackInputStream(in);
523                         }
524                         ((PushbackInputStream) in).unread(c2);
525                     }
526                     break loop;
527
528                 default:
529                     if (--room < 0) {
530                         buf = new char[offset + 128];
531                         room = buf.length - offset - 1;
532                         System.arraycopy(lineBuffer, 0, buf, 0, offset);
533                         lineBuffer = buf;
534                     }
535                     buf[offset++] = (char) c;
536                     break;
537             }
538         }
539         if ((c == -1) && (offset == 0)) {
540             return null;
541         }
542         return String.copyValueOf(buf, 0, offset);
543     }

```



FilterInputStream 与 装饰者模式

BufferedInputStream: 在read时，干脆多读一部分数据进来，放在内存里，等你每次操作流的时候，读取的数据直接从内存中就可以拿到，这就减少了io次数

```
public class BufferedInputStream extends FilterInputStream {
    //....省略部分源码
    private static int DEFAULT_BUFFER_SIZE = 8192;

    private static int MAX_BUFFER_SIZE = Integer.MAX_VALUE - 8;

    protected volatile byte buf[];

    public synchronized int read() throws IOException {
        if (pos >= count) {
            fill();
            if (pos >= count)
                return -1;
        }
        return getBufIfOpen()[pos++] & 0xff;
    }
}
```

每次调用read读取数据时，先查看要读取的数据是否在缓存中，如果在缓存中，直接从缓存中读取；如果不在缓存中，则调用fill方法，从InputStream中读取一定的存储到buf中。



```

private void fill() throws IOException {
    byte[] buffer = getBufIfOpen();
    if (markpos < 0)
        pos = 0;          /* no mark: throw away the buffer */
    else if (pos >= buffer.length) /* no room left in buffer */
        if (markpos > 0) { /* can throw away early part of the buffer */
            int sz = pos - markpos;
            System.arraycopy(buffer, markpos, buffer, 0, sz);
            pos = sz;
            markpos = 0;
        } else if (buffer.length >= marklimit) {
            markpos = -1; /* buffer got too big, invalidate mark */
            pos = 0;      /* drop buffer contents */
        } else if (buffer.length >= MAX_BUFFER_SIZE) {
            throw new OutOfMemoryError("Required array size too large");
        } else {          /* grow buffer */
            int nsz = (pos <= MAX_BUFFER_SIZE - pos) ?
                pos * 2 : MAX_BUFFER_SIZE;
            if (nsz > marklimit)
                nsz = marklimit;
            byte nbuf[] = new byte[nsz];
            System.arraycopy(buffer, 0, nbuf, 0, pos);
            if (!bufUpdater.compareAndSet(this, buffer, nbuf)) {
                // Can't replace buf if there was an async close.
                // Note: This would need to be changed if fill()
                // is ever made accessible to multiple threads.
                // But for now, the only way CAS can fail is via close.
                // assert buf == null;
                throw new IOException("Stream closed");
            }
            buffer = nbuf;
        }
    count = pos;
    int n = getInIfOpen().read(buffer, pos, buffer.length - pos);
    if (n > 0)
        count = n + pos;
}

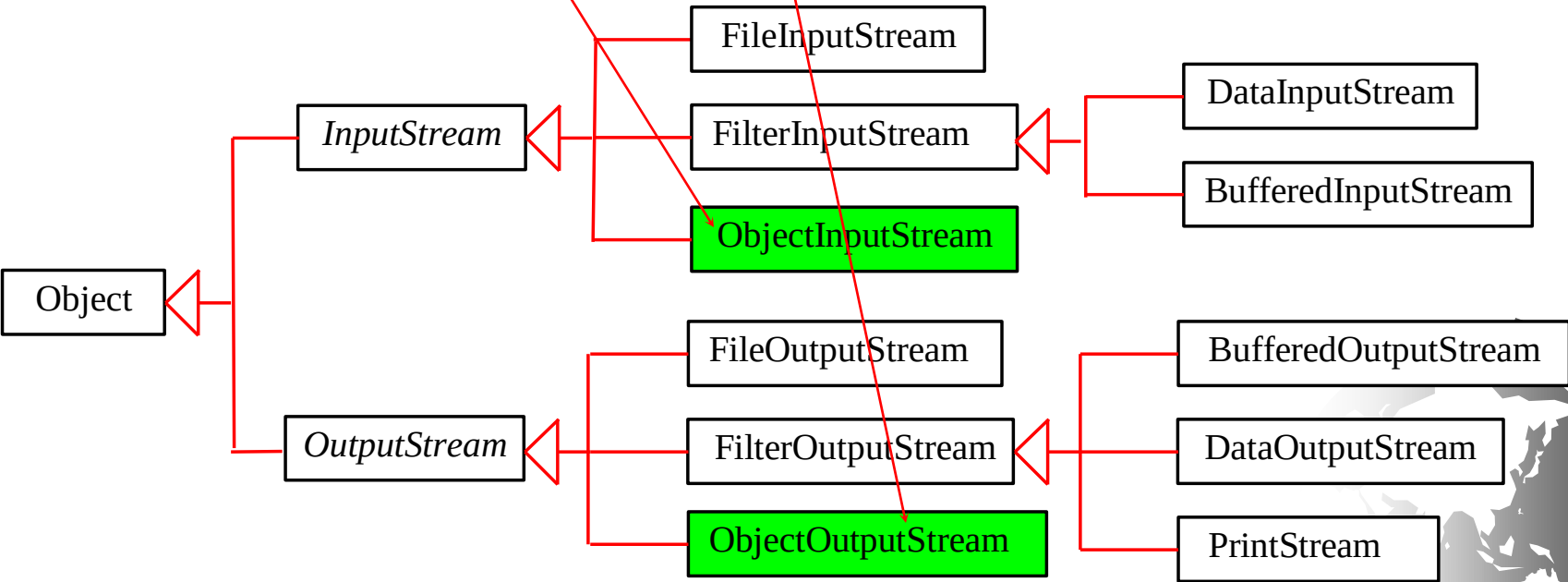
```



Object I/O

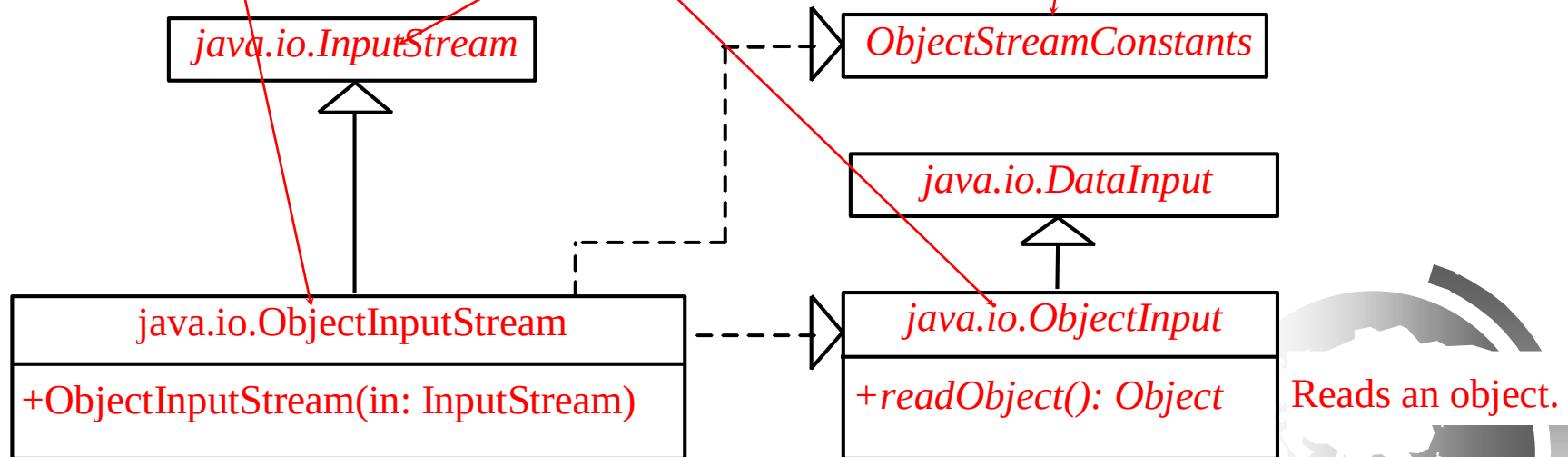
DataInputStream/DataOutputStream enables you to perform I/O for primitive type values and strings.

ObjectInputStream/ObjectOutputStream enables you to perform I/O for objects in addition for primitive type values and strings.



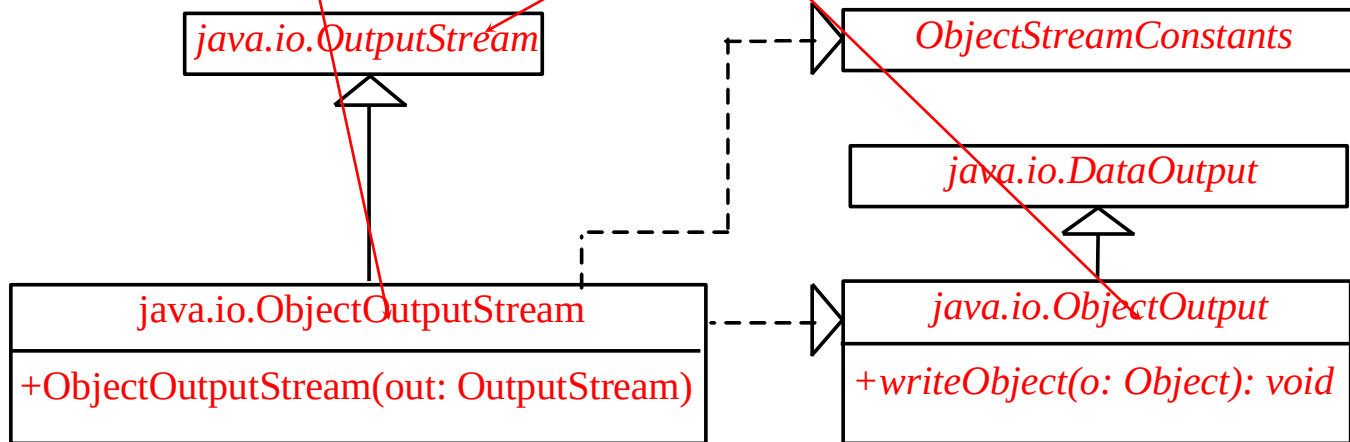
ObjectInputStream

ObjectInputStream extends InputStream and implements ObjectInput and ObjectStreamConstants.



ObjectOutputStream

ObjectOutputStream extends OutputStream and implements ObjectOutput and ObjectOutputStreamConstants.



Writes an object.



Using Object Streams

You may wrap an `ObjectInputStream/ObjectOutputStream` on any `InputStream/OutputStream` using the following constructors:

```
// Create an ObjectInputStream
```

```
public ObjectInputStream(InputStream in)
```

```
// Create an ObjectOutputStream
```

```
public ObjectOutputStream(OutputStream out)
```



TestObjectOutputStream

Run



TestObjectInputStream

Run

```

public class TestObjectOutputStream {
    public static void main(String[] args) throws IOException {
        try ( // Create an output stream for file object.dat
            ObjectOutputStream output =
                new ObjectOutputStream(new FileOutputStream("object.dat"));
        ) {
            // Write a string, double value, and object to the file
            output.writeUTF("John");
            output.writeDouble(85.5);
            output.writeObject(new java.util.Date());
        }
    }
}

```

```

public class TestObjectInputStream {
    public static void main(String[] args)
        throws ClassNotFoundException, IOException {
        try ( // Create an input stream for file object.dat
            ObjectInputStream input =
                new ObjectInputStream(new FileInputStream("object.dat"));
        ) {
            // Read a string, double value, and object from the file
            String name = input.readUTF();
            double score = input.readDouble();
            java.util.Date date = (java.util.Date) (input.readObject());
            System.out.println(name + " " + score + " " + date);
        }
    }
}

```



The Serializable Interface

Not all objects can be written to an output stream. Objects that can be written to an object stream is said to be *serializable*. **A serializable object is an instance of the `java.io.Serializable` interface.** So the class of a serializable object must **implement** `Serializable`.

The `Serializable` interface is a marker interface. It has no methods, so you don't need to add additional code in your class that implements `Serializable`.

Implementing this interface enables the Java serialization mechanism to automate the process of storing the objects and arrays.



The transient Keyword

If an object is an instance of Serializable, but it contains non-serializable instance data fields, can the object be serialized? The answer is no.

To enable the object to be serialized, you can use the **transient** keyword to mark these data fields to tell the JVM to ignore these fields when writing the object to an object stream.



The transient Keyword, cont.

Consider the following class:

```
public class Foo implements java.io.Serializable {  
    private int v1;  
    private static double v2;  
    private transient A v3 = new A();  
}  
class A { } // A is not serializable
```

When an object of the Foo class is serialized, only variable v1 is serialized. Variable v2 is not serialized because it is a static variable, and variable v3 is not serialized because it is marked transient. If v3 were not marked transient, a java.io.NotSerializableException would occur.

Serializing Arrays

An array is serializable if all its elements are serializable.

So an entire array can be saved using `writeObject` into a file and later restored using `readObject`. Here is an example that stores an array of five int values and an array of three strings, and reads them back to display on the console.



TestObjectStreamForArray

Run




```
public class TestObjectStreamForArray {
    public static void main(String[] args)
        throws ClassNotFoundException, IOException {
        int[] numbers = {1, 2, 3, 4, 5};
        String[] strings = {"John", "Susan", "Kim"};

        try ( // Create an output stream for file array.dat
            ObjectOutputStream output = new ObjectOutputStream(new
                FileOutputStream("array.dat", true));
        ) {
            // Write arrays to the object output stream
            output.writeObject(numbers);
            output.writeObject(strings);
        }

        try ( // Create an input stream for file array.dat
            ObjectInputStream input =
                new ObjectInputStream(new FileInputStream("array.dat"));
        ) {
            int[] newNumbers = (int[]) (input.readObject());
            String[] newStrings = (String[]) (input.readObject());

            // Display arrays
            for (int i = 0; i < newNumbers.length; i++)
                System.out.print(newNumbers[i] + " ");
            System.out.println();

            for (int i = 0; i < newStrings.length; i++)
                System.out.print(newStrings[i] + " ");

        }
    }
}
```

1. 反序列化后的对象，需要调用构造函数重新构造吗？

答案：不需要。对于Serializable对象，对象完全以它存储的二进制位作为基础来构造，而不调用构造器。

```
package test.serializable;

import java.io.Serializable;
import java.util.Date;

/**
 *
 * @author chenfei
 *
 * 用于测试序列化时的deep copy
 *
 */
public class House implements Serializable {
    private static final long serialVersionUID = -6091530420906090649L;

    private Date date = new Date(); //记录当前的时间

    public String toString() {
        return "House:" + super.toString() + ".Create Time is:" + date;
    }
}
```



```
package test.serializable;

import java.io.Serializable;

public class Animal implements Serializable {
    private static final long serialVersionUID = -213221189192962074L;

    private String name;

    private House house;

    public Animal(String name , House house) {
        this.name = name;
        this.house = house;
        System.out.println("调用了构造器");
    }

    public String toString() {
        return name + "[" + super.toString() + "]" + house;
    }

}
```

```
public static void main(String[] args) throws IOException, ClassNotFoundException {  
    House house = new House();  
    System.out.println("序列化前");  
    Animal animal = new Animal("test",house);  
    ByteArrayOutputStream out = new ByteArrayOutputStream();  
    ObjectOutputStream oos = new ObjectOutputStream(out);  
    oos.writeObject(animal);  
    oos.flush();  
    oos.close();  
  
    System.out.println("反序列化后");  
    ByteArrayInputStream in = new ByteArrayInputStream(out.toByteArray());  
    ObjectInputStream ois = new ObjectInputStream(in);  
    Animal animal1 = (Animal)ois.readObject();  
    ois.close();  
}
```

运行结果如下所示:

序列化前
调用了构造器
反序列化后

问题：序列前的对象与序列化后的对象是什么关系？
是"=="还是equal？是浅复制还是深复制？



```

public class Animal implements Serializable{

    private String name;
    public Animal(){
        System.out.println("默认构造函数");
    }
    public Animal(String s){
        name = s;
        System.out.println("带参构造函数");
    }
    public static void main(String[] args) throws IOException, ClassNotFoundException{
        Animal a = new Animal("cat");
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(out);
        oos.writeObject(a);
        oos.flush();
        oos.close();

        ByteArrayInputStream in = new ByteArrayInputStream(out.toByteArray());
        ObjectInputStream ois = new ObjectInputStream(in);
        Animal a2 = (Animal)ois.readObject();
        ois.close();
    }
}

```

 Problems
  Javadoc
  Declaration
  Console

<terminated> Animal (1) [Java Application]

带参构造函数



2. 序列前的对象与序列化后的对象是什么关系？ 是"=="还是 equal？是浅复制还是深复制？

```
public static void main(String[] args) throws IOException, ClassNotFoundException {  
    House house = new House();  
    System.out.println("序列化前");  
    Animal animal = new Animal("test",house);  
    System.out.println(animal);  
    ByteArrayOutputStream out = new ByteArrayOutputStream();  
    ObjectOutputStream oos = new ObjectOutputStream(out);  
    oos.writeObject(animal);  
    oos.writeObject(animal); //在写一次，看对象是否是一样，  
    oos.flush();  
    oos.close();  
  
    ByteArrayOutputStream out2 = new ByteArrayOutputStream(); //换一个输出流  
    ObjectOutputStream oos2 = new ObjectOutputStream(out2);  
    oos2.writeObject(animal);  
    oos2.flush();  
    oos2.close();  
  
    System.out.println("反序列化后");  
    ByteArrayInputStream in = new ByteArrayInputStream(out.toByteArray());  
    ObjectInputStream ois = new ObjectInputStream(in);  
    Animal animal1 = (Animal)ois.readObject();  
    Animal animal2 = (Animal)ois.readObject();  
    ois.close();  
  
    ByteArrayInputStream in2 = new ByteArrayInputStream(out2.toByteArray());  
    ObjectInputStream ois2 = new ObjectInputStream(in2);  
    Animal animal3 = (Animal)ois2.readObject();  
    ois2.close();  
  
    System.out.println("out流: " + animal1);  
    System.out.println("out流: " + animal2);  
    System.out.println("out2流: " + animal3);  
  
    System.out.println("测试序列化前后的对象 == : "+ (animal==animal1));  
    System.out.println("测试序列化后同一流的对象: "+ (animal1 == animal2));  
    System.out.println("测试序列化后不同流的对象==:" + (animal1==animal3));  
}
```

2. 序列前的对象与序列化后的对象是什么关系？是("==" 还是equal？ 是浅复制还是深复制？)

序列化前

调用了构造器

```
test[test.serializable.Animal@bb7465']House:test.serializable.House@d6c16c.Create Time is:Sat Apr 06 00:11:30 CST 2013
```

反序列化后

```
out流: test[test.serializable.Animal@4f80d6']House:test.serializable.House@193722c.Create Time is:Sat Apr 06 00:11:30 CST 2013
```

```
out流: test[test.serializable.Animal@4f80d6']House:test.serializable.House@193722c.Create Time is:Sat Apr 06 00:11:30 CST 2013 (与上面的相同)
```

```
out2流: test[test.serializable.Animal@12cc95d']House:test.serializable.House@157fb52.Create Time is:Sat Apr 06 00:11:30 CST 2013 (与上面只是值相同，但是地址不一样。)
```

```
测试序列化前后的对象 == : false
```

```
测试序列化后同一流的对象: true
```

```
测试序列化后不同流的对象==: false
```

答案：深复制，反序列化还原后的对象地址与原来的的地址不同。

通过序列化操作，我们可以实现对任何可Serializable对象的”深度复制（deep copy）”——这意味着我们复制的是整个对象网，而不仅仅是基本对象及其引用。对于同一流的对象，他们的地址是相同，说明他们是同一个对象，但是与其他流的对象地址却不相同。

3. serialVersionUID 的作用？

Serializable接口的代价一：若一个类实现Serializable接口，会大大降低了“改变这个类的实现”的灵活性。

如果不设计一种自定义的序列化形式，而是采用默认的序列化形式，则会被束缚在该类最初的内部表示法上。就是说，若之后改变该类的内部表示法，结果会导致不兼容。用旧版本来序列化一个类，而用新版本来反序列化，会导致程序失效。

凡是实现Serializable接口的类都有一个表示序列化版本标识符的静态long类型变量：`private static final long serialVersionUID;`

如果没有设置这个值，在序列化一个对象之后，改动了该类的字段或者方法名之类的，再反序列化想取出之前的那个对象时就可能会抛出异常，因为serialVersionUID是根据类名、接口名、成员方法及属性等来生成一个64位的哈希字段，当修改后的类去反序列化时，该类的serialVersionUID值和之前保存在文件中的serialVersionUID值不一致，所以就会抛出异常。


```

public static void main(String[] args) throws Exception {
    SerializeCustomer(); // 序列化Customer对象
    Customer customer = DeserializeCustomer(); // 反序列Customer对象
    System.out.println(customer);
}

private static void SerializeCustomer() throws FileNotFoundException,
IOException {
    Customer customer = new Customer("gacl",25);
    // ObjectOutputStream 对象输出流
    ObjectOutputStream oo = new ObjectOutputStream(new FileOutputStream(
        new File("D:/TEMP/Customer.txt")));
    oo.writeObject(customer);
    System.out.println("Customer对象序列化成功!");
    oo.close();
}

private static Customer DeserializeCustomer() throws Exception, IOException {
    ObjectInputStream ois = new ObjectInputStream(new FileInputStream(
        new File("D:/TEMP/Customer.txt")));
    Customer customer = (Customer) ois.readObject();
    System.out.println("Customer对象反序列化成功!");
    return customer;
}

```

```

public class Customer implements Serializable {
    private String name ;
    private int age;
    public Customer(String name, int age){
        this.name = name;
        this.age = age;
    }
    @Override
    public String toString() {
        return "name=" + name + ", age=" + age;
    }
}

```

Customer对象序列化成功!
Customer对象反序列化成功!
name=gacl, age=25

```

public class Customer implements Serializable {
    private String name ;
    private int age;
    private String sex;
    public Customer(String name, int age){
        this.name = name;
        this.age = age;
    }
    public Customer(String name,int age, String sex){
        this.name = name;
        this.age = age;
        this.sex = sex;
    }
    @Override
    public String toString() {
        return "name=" + name + ", age=" + age;
    }
}

```

修改Customer后，对原来的文件进行反序列化

```

public static void main(String[] args) throws Exception {
    // SerializeCustomer();// 序列化Customer对象
    Customer customer = DeserializeCustomer();// 反序列化Customer对象
    System.out.println(customer);
}

```

Exception in thread "main" [java.io.InvalidClassException](#): testForSerializable.Customer; local class incompatible
 at java.io.ObjectStreamClass.initNonProxy([ObjectStreamClass.java:621](#))
 at java.io.ObjectInputStream.readNonProxyDesc([ObjectInputStream.java:1623](#))
 at java.io.ObjectInputStream.readClassDesc([ObjectInputStream.java:1518](#))
 at java.io.ObjectInputStream.readOrdinaryObject([ObjectInputStream.java:1774](#))
 at java.io.ObjectInputStream.readObject0([ObjectInputStream.java:1351](#))
 at java.io.ObjectInputStream.readObject([ObjectInputStream.java:371](#))
 at testForSerializable.TestSerialVersionUID.DeserializeCustomer([TestSerialVersionUID.java:32](#))
 at testForSerializable.TestSerialVersionUID.main([TestSerialVersionUID.java:15](#))

```

public class Customer implements Serializable {
    private static final long serialVersionUID = -5182532647273106745L;
    private String name ;
    private int age;
    // private String sex;
    public Customer(String name, int age){
        this.name = name;
        this.age = age;
    }
    /* public Customer(String name,int age, String sex){
        this.name = name;
        this.age = age;
        this.sex = sex;
    }*/
    @Override
    public String toString() {
        return "name=" + name + ", age=" + age;
    }
}

```

```

public static void main(String[] args) throws Exception {
    SerializeCustomer();// 序列化Customer对象
    Customer customer = DeserializeCustomer();// 反序列化Customer对象
    System.out.println(customer);
}

```

Customer对象序列化成功!
 Customer对象反序列化成功!
 name=gac1, age=25



```

public class Customer implements Serializable {
    private static final long serialVersionUID = -5182532647273106745L;
    private String name ;
    private int age;
    private String sex;
    public Customer(String name, int age){
        this.name = name;
        this.age = age;
    }
    public Customer(String name,int age, String sex){
        this.name = name;
        this.age = age;
        this.sex = sex;
    }
    @Override
    public String toString() {
        return "name=" + name + ", age=" + age;
    }
}

```

修改Customer后，对原来的文件进行反序列化

```

public static void main(String[] args) throws Exception {
    // SerializeCustomer();// 序列化Customer对象
    Customer customer = DeserializeCustomer();// 反序列Customer对象
    System.out.println(customer);
}

```

Customer对象反序列化成功!
name=gac1, age=25



4. writeObject 和 readObject

当 ObjectOutputStream 对一个 Customer 对象进行序列化时，如果该对象具有 writeObject() 方法，那么就会执行这一方法，否则就按默认方式序列化。在该对象的 writeObject() 方法中，可以先调用 ObjectOutputStream 的 defaultWriteObject() 方法，使得对象输出流先执行默认的序列化操作。同理可得出反序列化的情况，不过这次是 defaultReadObject() 方法。

```
public class Person implements Serializable {  
    ...  
    transient private Integer age = null;  
    ...  
  
    private void writeObject(ObjectOutputStream out) throws IOException {  
        out.defaultWriteObject();  
        out.writeInt(age);  
    }  
  
    private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {  
        in.defaultReadObject();  
        age = in.readInt();  
    }  
}
```

虽然 age 为 transient，不会被默认序列化，但可以用 writeObject 和 readObject 进行序列化



5. Externalizable

Java默认的序列化机制非常简单，而且序列化后的对象不需要再次调用构造器重新生成，但是在实际中，我们会希望对象的某一部分不需要被序列化，或者说一个对象被还原之后，其内部的某些子对象需要重新创建，从而不必将该子对象序列化。在这些情况下，我们可以考虑实现Externalizable接口从而代替Serializable接口来对序列化过程进行控制（通过transient的方式更简单）。

相当于完全自己控制序列化和反序列化，之前基于Serializable接口的序列化机制就将失效。

Externalizable接口extends Serializable接口，而且在其基础上增加了两个方法：writeExternal()和readExternal()。这两个方法会在序列化和反序列化还原的过程中被自动调用，以便执行一些特殊的操作。

```
public interface Externalizable extends java.io.Serializable {  
  
    void writeExternal(ObjectOutput out) throws IOException;  
  
    void readExternal(ObjectInput in) throws IOException, ClassNotFoundException;  
}
```

```
public class Blip implements Externalizable {
```

```
    private int i ;
```

```
    private String s;//没有初始化
```

```
    public Blip() {  
        //默认构造函数必须有, 而且必须是public  
        System.out.println("Blip默认构造函数");  
    }
```

```
    public Blip(String s ,int i) {  
        //s,i只是在带参数的构造函数中进行初始化。  
        System.out.println("Blip带参数构造函数");  
        this.s = s;  
        this.i = i;  
    }
```

```
    public String toString() {  
        return s + i ;  
    }
```

```
    @Override
```

```
    public void readExternal(ObjectInput in) throws IOException,  
        ClassNotFoundException {  
        System.out.println("调用readExternal () 方法");
```

```
        s = (String)in.readObject();//在反序列化时, 需要初始化s和i, 否则只是调用默认构造函数, 得不到s和i的值  
        i = in.readInt();
```

```
    }
```

```
    @Override
```

```
    public void writeExternal(ObjectOutput out) throws IOException {  
        System.out.println("调用writeExternal () 方法");
```

```
        out.writeObject(s); //如果我们不将s和i的值写入的话, 那么在反序列化的时候, 就不会得到这些值。  
        out.writeInt(i);
```

```
    }
```

```
}
```



```
public class ExternalizableTest {

    /**
     * @param args
     * @throws IOException
     * @throws ClassNotFoundException
     */
    public static void main(String[] args) throws IOException, ClassNotFoundException {
        System.out.println("序列化之前");
        Blip b = new Blip("This String is " , 47);
        System.out.println(b);

        System.out.println("序列化操作, writeObject");
        ByteArrayOutputStream out = new ByteArrayOutputStream();
        ObjectOutputStream oos = new ObjectOutputStream(out);
        oos.writeObject(b);
        System.out.println("反序列化之后, readObject");
        ByteArrayInputStream in = new ByteArrayInputStream(out.toByteArray());
        ObjectInputStream ois = new ObjectInputStream(in);
        Blip bb = (Blip)ois.readObject();
        System.out.println(bb);
    }
}
```

序列化之前

Blip带参数构造函数

This String is 47

序列化操作, writeObject
调用writeExternal () 方法
反序列化之后, readObject

Blip默认构造函数

调用readExternal () 方法
This String is 47

1) 如果我们只修改writeExternal () 方法如下:

```
@Override
    public void writeExternal(ObjectOutput out) throws IOException {
        System.out.println("调用writeExternal () 方法");
        //        out.writeObject(s);
        //        out.writeInt(i);
    }
```

那么运行的结果为:



序列化之前
Blip带参数构造函数
This String is 47
序列化操作, writeObject
调用writeExternal () 方法
反序列化之后, readObject
Blip默认构造函数
调用readExternal () 方法

```
Exception in thread "main" java.io.OptionalDataException
    at java.io.ObjectInputStream.readObject0(ObjectInputStream.java:1349)
    at java.io.ObjectInputStream.readObject(ObjectInputStream.java:351)
    at test.serializable.Blip.readExternal(Blip.java:34)
    at java.io.ObjectInputStream.readExternalData(ObjectInputStream.java:1792)
    at java.io.ObjectInputStream.readOrdinaryObject(ObjectInputStream.java:1751)
    at java.io.ObjectInputStream.readObject0(ObjectInputStream.java:1329)
    at java.io.ObjectInputStream.readObject(ObjectInputStream.java:351)
    at test.serializable.ExternalizableTest.main(ExternalizableTest.java:28)
```



原因是因为, 我们在ObjectOutputStream中没有writeObject,而在ObjectInputStream中readObject导致的



2) 如果我们修改writeExternal () 方法如下:

```
@Override
    public void writeExternal(ObjectOutput out) throws IOException {
        System.out.println("调用writeExternal () 方法");
        out.writeObject("自己定义的");
        out.writeInt(250);
    }
```

那么运行的结果为:



序列化之前

Blip带参数构造函数

This String is 47

序列化操作, writeObject

调用writeExternal () 方法

反序列化之后, readObject

Blip默认构造函数

调用readExternal () 方法

自己定义的250



反序列化后得到的s和i是我们在writeExternal () 中自定义的数据



3) 如果我们只是修改readExternal()方法



```
@Override
public void readExternal(ObjectInput in) throws IOException,
    ClassNotFoundException {
    System.out.println("调用readExternal () 方法");
    //      s = (String)in.readObject();
    //      i = in.readInt();
}
```



那么运行的结果为:



序列化之前
Blip带参数构造函数
This String is 47
序列化操作, writeObject
调用writeExternal () 方法
反序列化之后, readObject
Blip默认构造函数
调用readExternal () 方法
null0



4) 如果我们删除Blip的默认构造函数，或者将其权限不设置为public

```
// public Blip() {  
//     //默认构造函数必须有，而且必须是public  
//     System.out.println("Blip默认构造函数");  
// }  
// or  
Blip() {  
    //默认构造函数必须有，而且必须是public  
    System.out.println("Blip默认构造函数");  
}
```

运行结果如下：



序列化之前

Blip带参数构造函数

This String is 47

序列化操作, writeObject

调用writeExternal () 方法

反序列化之后, readObject

```
Exception in thread "main" java.io.InvalidClassException: test.serializable.Blip; test.serializable.Blip; no valid constructor  
    at java.io.ObjectStreamClass.checkDeserialize(ObjectStreamClass.java:713)  
    at java.io.ObjectInputStream.readOrdinaryObject(ObjectInputStream.java:1733)  
    at java.io.ObjectInputStream.readObject0(ObjectInputStream.java:1329)  
    at java.io.ObjectInputStream.readObject(ObjectInputStream.java:351)  
    at test.serializable.ExternalizableTest.main(ExternalizableTest.java:28)  
Caused by: java.io.InvalidClassException: test.serializable.Blip; no valid constructor  
    at java.io.ObjectStreamClass.<init>(ObjectStreamClass.java:471)  
    at java.io.ObjectStreamClass.lookup(ObjectStreamClass.java:310)  
    at java.io.ObjectOutputStream.writeObject0(ObjectOutputStream.java:1106)  
    at java.io.ObjectOutputStream.writeObject(ObjectOutputStream.java:326)  
    at test.serializable.ExternalizableTest.main(ExternalizableTest.java:24)
```

必须有权限为public的默认的构造器



Externalizable vs Serializable

(1) Serializable 是标识接口；Externalizable 接口继承于Serializable，实现该接口，需要重写readExternal和writeExternal方法

(2) Serializable提供了两种方式进行对象的序列化：采用默认序列化方式，将非transient和非static的属性进行序列化；编写readObject和writeObject完成部分属性的序列化。

Externalizable 接口的序列化，需要重写writeExternal和readExternal方法，并且在方法中编写相关的逻辑完成序列化和反序列化。

(3) Externalizable接口的实现方式一定要有默认**的无参构造函数**，如果，没有无参构造函数，反序列化会报错。对于一个Externalizable对象，对象的默认构造函数都会被调用（包括哪些在定义时已经初始化的字段），然后调用readExternal()

(4) 采用Externalizable无需产生序列化ID（serialVersionUID），而Serializable接口则需要。

(5) 相比较Serializable, Externalizable序列化、反序列更加快速，占用相比较小的内存



6. 对单例模式的影响

单例模式的序列化

```
public class Customer implements Serializable {
    private static final long serialVersionUID = -5182532647273106745L;
    private String name ;
    private int age;
    private String sex;
    private static class InstanceHolder {
        private static final Customer instatnce = new Customer("John", 31);
    }

    public static Customer getInstance() {
        return InstanceHolder.instatnce;
    }

    private Customer(String name, int age){
        this.name = name;
        this.age = age;
    }
    private Customer(String name,int age, String sex){
        this.name = name;
        this.age = age;
        this.sex = sex;
    }
    @Override
    public String toString() {
        return "name=" + name + ", age=" + age;
    }
}
```

```
public class TestSerialSingleton {  
  
    public static void main(String[] args) throws IOException, ClassNotFoundException {  
        File file = new File("D:/TEMP/Custom.txt");  
        ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(file));  
        out.writeObject(Customer.getInstance()); // 保存单例对象  
        out.close();  
  
        ObjectInputStream oin = new ObjectInputStream(new FileInputStream(file));  
        Object newCustomer = oin.readObject();  
        oin.close();  
        System.out.println(newCustomer);  
  
        System.out.println(Customer.getInstance() == newCustomer);  
  
    }  
}
```

破坏了单例性！

name=John, age=31
false

为了能在序列化过程仍能保持单例的特性，可以在Customer类中添加一个**readResolve()**方法，在该方法中直接返回Customer的单例对象

```
private Object readResolve() throws ObjectStreamException {  
    return InstanceHolder.instance;  
}
```

```
public class TestSerialSingleton {  
  
    public static void main(String[] args) throws IOException, ClassNotFoundException {  
        File file = new File("D:/TEMP/Customer.txt");  
        ObjectOutputStream out = new ObjectOutputStream(new FileOutputStream(file));  
        out.writeObject(Customer.getInstance()); // 保存单例对象  
        out.close();  
  
        ObjectInputStream oin = new ObjectInputStream(new FileInputStream(file));  
        Object newCustomer = oin.readObject();  
        oin.close();  
        System.out.println(newCustomer);  
  
        System.out.println(Customer.getInstance() == newCustomer);  
    }  
}
```

name=John, age=31
true

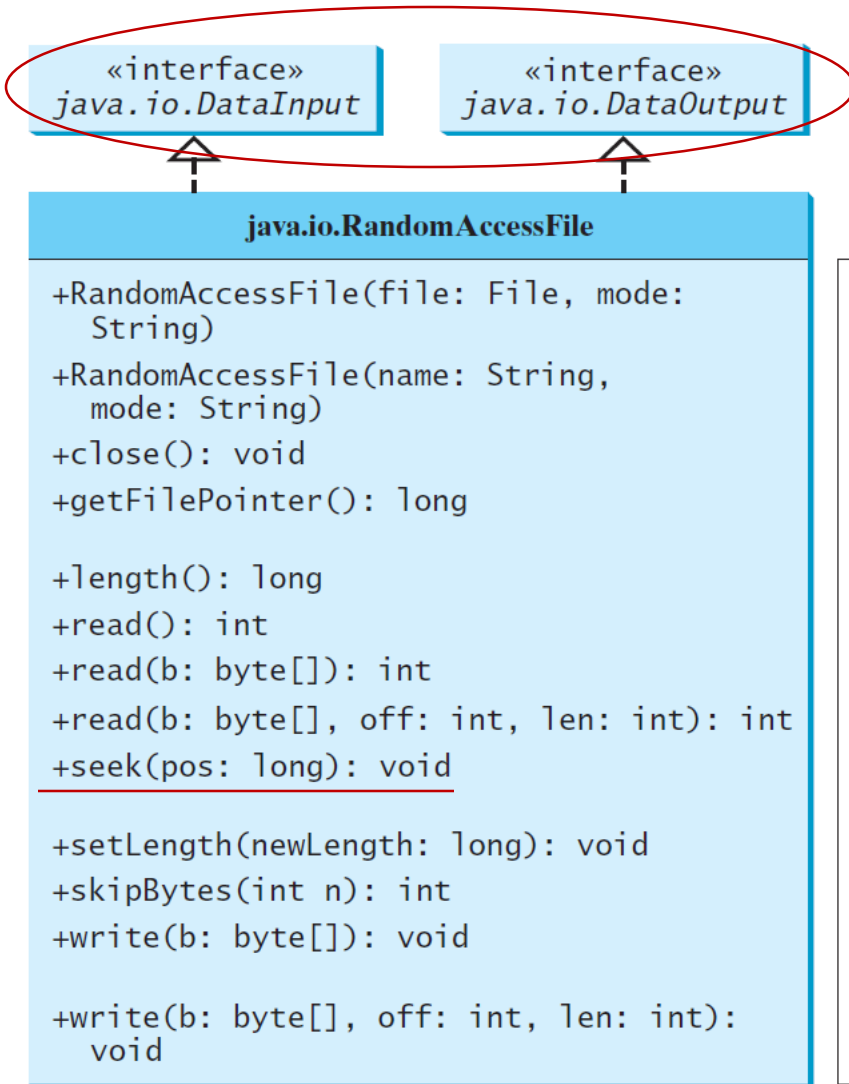
无论是实现Serializable接口，或是Externalizable接口，当从I/O流中读取对象时，readResolve()方法都会被调用到。实际上就是用readResolve()中返回的对象直接替换在反序列化过程中创建的对象。

Random Access Files

All of the streams you have used so far are known as *read-only* or *write-only* streams. The external files of these streams are *sequential files* that cannot be updated without creating a new file. It is often necessary to modify files or to insert new records into files. Java provides the `RandomAccessFile` class to allow a file to be *read from and write to at random locations*.



RandomAccessFile



Creates a `RandomAccessFile` stream with the specified `File` object and mode.

Creates a `RandomAccessFile` stream with the specified file name string and mode.

Closes the stream and releases the resource associated with it.

Returns the offset, in bytes, from the beginning of the file to where the next `read` or `write` occurs.

Returns the number of bytes in this file.

Reads a byte of data from this file and returns `-1` at the end of stream.

Reads up to `b.length` bytes of data from this file into an array of bytes.

Reads up to `len` bytes of data from this file into an array of bytes.

Sets the offset (in bytes specified in `pos`) from the beginning of the stream to where the next `read` or `write` occurs.

Sets a new length for this file.

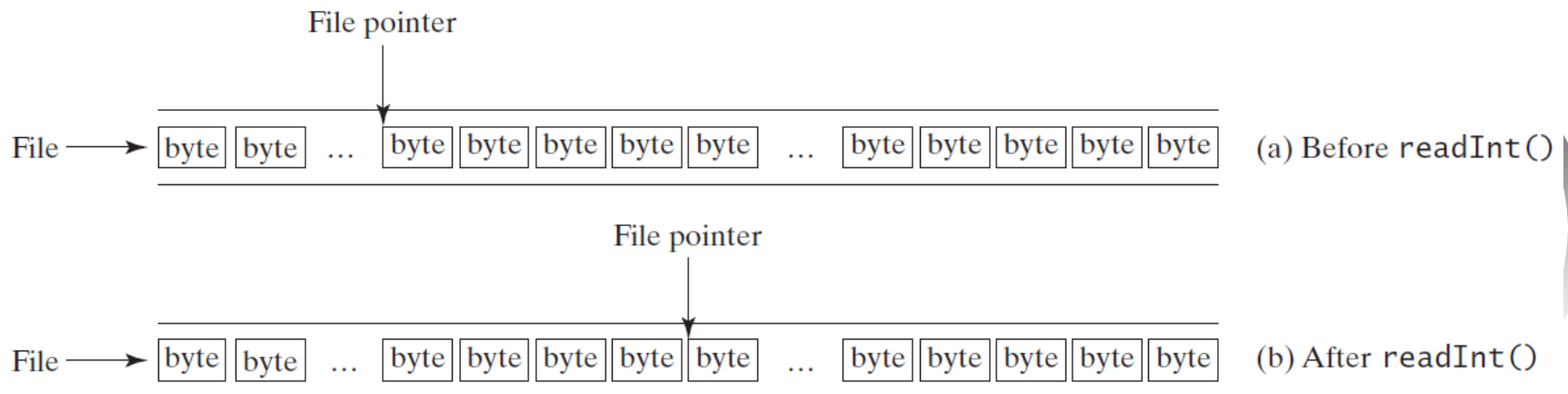
Skips over `n` bytes of input.

Writes `b.length` bytes from the specified byte array to this file, starting at the current file pointer.

Writes `len` bytes from the specified byte array, starting at offset `off`, to this file.

File Pointer

A random access file consists of a sequence of bytes. **There is a special marker called *file pointer* that is positioned at one of these bytes.** A read or write operation takes place at the location of the file pointer. When a file is opened, the file pointer sets at the beginning of the file. When you read or write data to the file, the file pointer moves forward to the next data. For example, if you read an int value using `readInt()`, the JVM reads four bytes from the file pointer and now the file pointer is four bytes ahead of the previous location.



RandomAccessFile Methods

Many methods in `RandomAccessFile` are the same as those in `DataInputStream` and `DataOutputStream`. For example, `readInt()`, `readLong()`, `writeDouble()`, `readLine()`, `writeInt()`, and `writeLong()` can be used in data input stream or data output stream as well as in `RandomAccessFile` streams.



RandomAccessFile Methods, cont.

`void seek(long pos) throws IOException;`

Sets the offset from the beginning of the `RandomAccessFile` stream to where the next read or write occurs.

`long getFilePointer() throws IOException;`

Returns the current offset, in bytes, from the beginning of the file to where the next read or write occurs.



RandomAccessFile Methods, cont.

```
long length() IOException
```

Returns the length of the file.

```
final void writeChar(int v) throws  
IOException
```

Writes a character to the file as a two-byte Unicode, with the high byte written first.

```
final void writeChars(String s)  
throws IOException
```

Writes a string to the file as a sequence of characters.



RandomAccessFile Constructor

```
RandomAccessFile raf =  
    new RandomAccessFile("test.dat", "rw");  
    // allows read and write
```

```
RandomAccessFile raf =  
    new RandomAccessFile("test.dat", "r");  
    // read only
```



A Short Example on RandomAccessFile



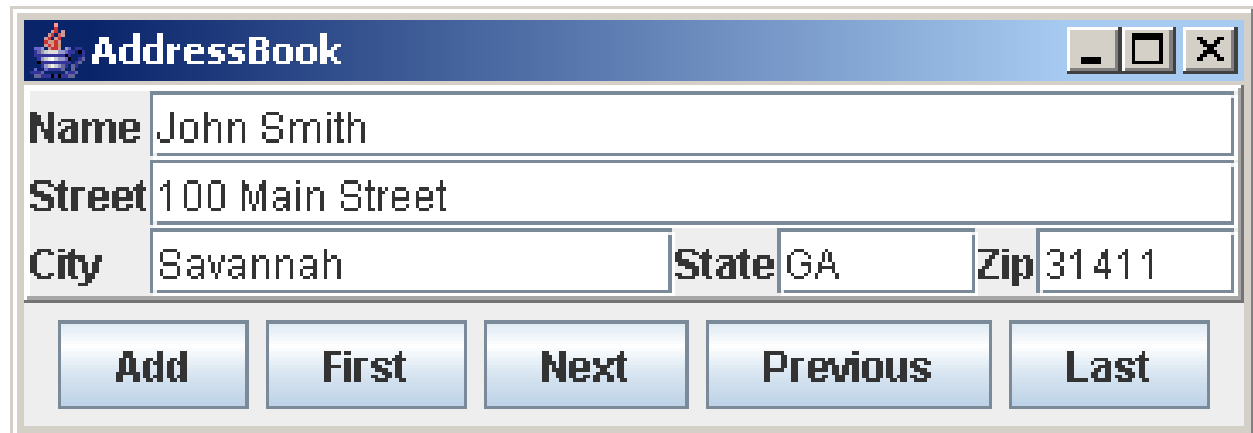
TestRandomAccessFile

Run



Case Studies: Address Book

Now let us use `RandomAccessFile` to create a useful project for storing and viewing an address book. The *Add* button stores a new address to the end of the file. The *First*, *Next*, *Previous*, and *Last* buttons retrieve the first, next, previous, and last addresses from the file, respectively.



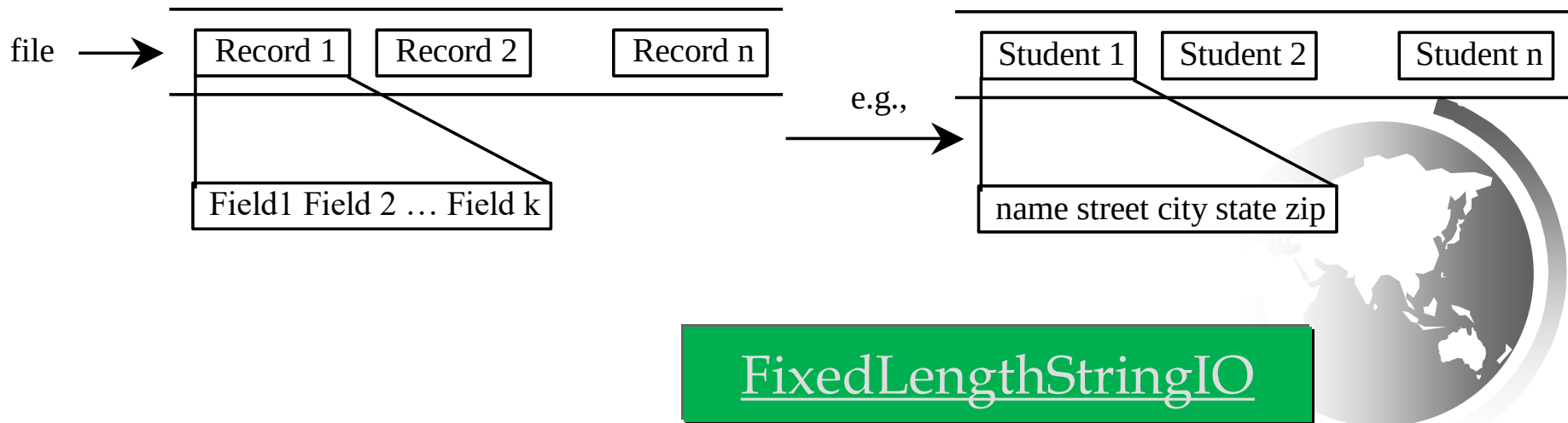
The screenshot shows a Java Swing window titled "AddressBook". It contains a form with the following fields and values:

Name	John Smith				
Street	100 Main Street				
City	Savannah	State	GA	Zip	31411

Below the form are five buttons: Add, First, Next, Previous, and Last.

Fixed Length String I/O

Random access files are often used to process files of records. For convenience, **fixed-length records are used in random access files so that a record can be located easily**. A record consists of a fixed number of fields. A field can be a string or a primitive data type. A string in a fixed-length record has a maximum size. If a string is smaller than the maximum size, the rest of the string is **padded** with blanks.



Address Implementation

The rest of the work can be summarized in the following steps:

Create the user interface.

Add a record to the file.

Read a record from the file.

Write the code to implement the button actions.



AddressBook

Run

```

/** Write a record at the end of the file */
public void writeAddress() {
    try {
        raf.seek(raf.length());
        FixedLengthStringIO.writeFixedLengthString(
            jtfName.getText(), NAME_SIZE, raf);
        FixedLengthStringIO.writeFixedLengthString(
            jtfStreet.getText(), STREET_SIZE, raf);
        FixedLengthStringIO.writeFixedLengthString(
            jtfCity.getText(), CITY_SIZE, raf);
        FixedLengthStringIO.writeFixedLengthString(
            jtfState.getText(), STATE_SIZE, raf);
        FixedLengthStringIO.writeFixedLengthString(
            jtfZip.getText(), ZIP_SIZE, raf);
    }
    catch (IOException ex) {
        ex.printStackTrace();
    }
}

/** Read a record at the specified position */
public void readAddress(long position) throws IOException {
    raf.seek(position);
    String name = FixedLengthStringIO.readFixedLengthString(
        NAME_SIZE, raf);
    String street = FixedLengthStringIO.readFixedLengthString(
        STREET_SIZE, raf);
    String city = FixedLengthStringIO.readFixedLengthString(
        CITY_SIZE, raf);
    String state = FixedLengthStringIO.readFixedLengthString(
        STATE_SIZE, raf);
    String zip = FixedLengthStringIO.readFixedLengthString(
        ZIP_SIZE, raf);
}

```

```

public class FixedLengthStringIO {
    /** Read fixed number of characters from a DataInput stream */
    public static String readFixedLengthString(int size,
        DataInput in) throws IOException {
        // Declare an array of characters
        char[] chars = new char[size];

        // Read fixed number of characters to the array
        for (int i = 0; i < size; i++)
            chars[i] = in.readChar();

        return new String(chars);
    }

    /** Write fixed number of characters to a DataOutput stream */
    public static void writeFixedLengthString(String s, int size,
        DataOutput out) throws IOException {
        char[] chars = new char[size];

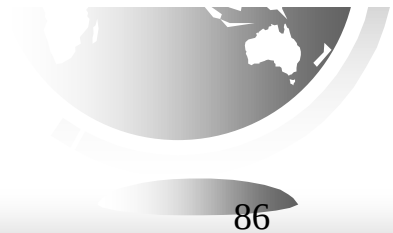
        // Fill in string with characters
        s.getChars(0, Math.min(s.length(), size), chars, 0);

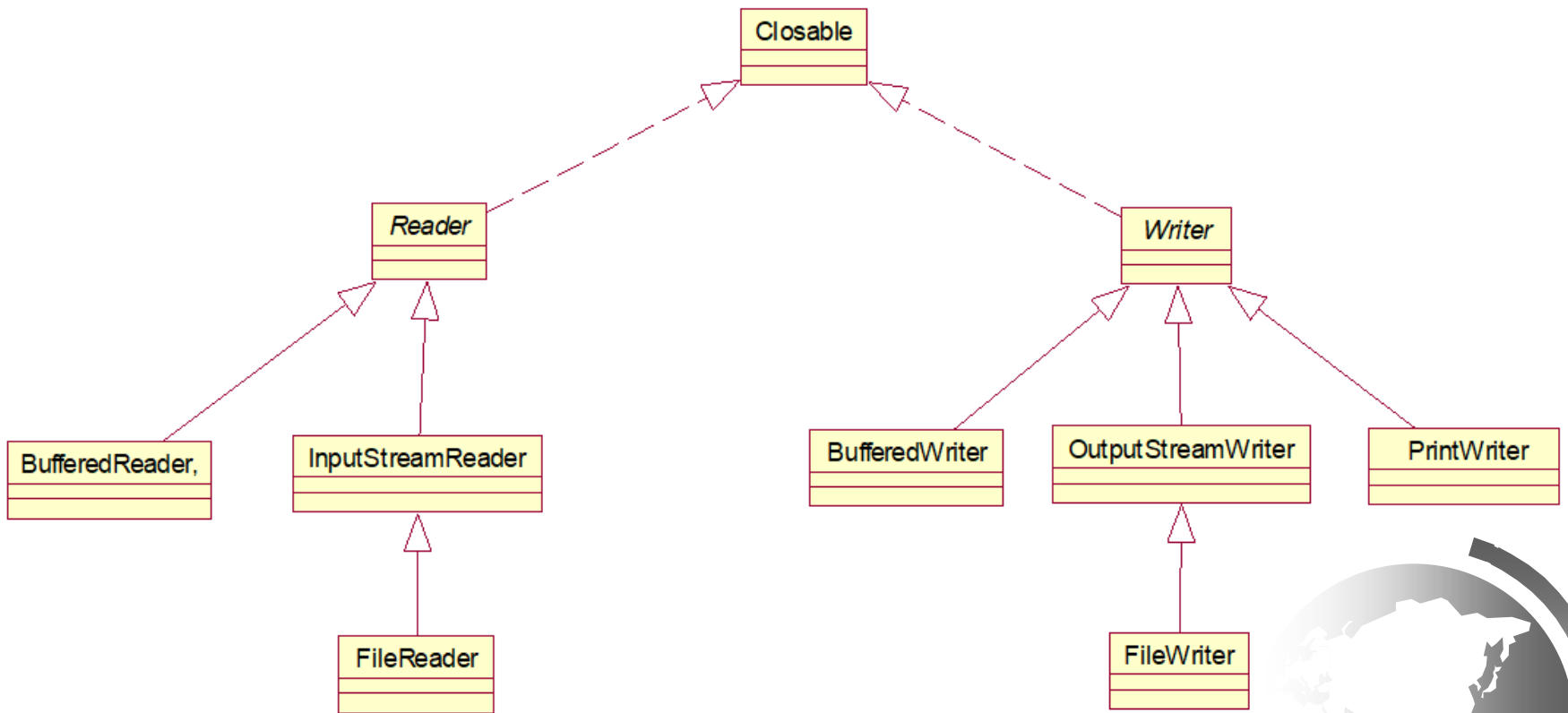
        // Fill in blank characters in the rest of the array
        for (int i = Math.min(s.length(), size); i < chars.length; i++)
            chars[i] = ' ';

        // Create and write a new string padded with blank characters
        out.writeChars(new String(chars));
    }
}

```

Advanced Text IO





字符流基类

Reader:读取字符流的抽象类.

常用方法:

// 读取单个字符

int read()

// 将字符读入数组

int read(char[] cbuf)

// 将字符读入数组的某一部分

abstract int read(char[] cbuf, int off, int len)

// 跳过字符

long skip(long n)

// 关闭该流并释放与之关联的所有资源

abstract void close()



Writer:写入字符流的抽象类.

常用方法:

```
// 写入字符数组
void write(char[] cbuf)
// 写入字符数组的某一部分
abstract void write(char[] cbuf, int off, int len)
// 写入单个字符
void write(int c)
// 写入字符串
void write(String str)
// 写入字符串的某一部分
void write(String str, int off, int len)

// 将指定字符添加到此 writer
Writer append(char c)
// 将指定字符序列添加到此 writer
Writer append(CharSequence csq)
// 将指定字符序列的子序列添加到此 writer.Appendable
Writer append(CharSequence csq, int start, int end)

// 关闭此流, 但要先刷新它
abstract void close()
// 刷新该流的缓冲
abstract void flush()
```



字符转换流

InputStreamReader: **字节流转字符流**，它使用的字符集可以由名称指定或显式给定，否则将接受平台默认的字符集。

构造方法:

```
// 创建一个使用默认字符集的 InputStreamReader
InputStreamReader(InputStream in)

// 创建使用给定字符集的 InputStreamReader
InputStreamReader(InputStream in, Charset cs)

// 创建使用给定字符集解码器的 InputStreamReader
InputStreamReader(InputStream in, CharsetDecoder dec)

// 创建使用指定字符集的 InputStreamReader
InputStreamReader(InputStream in, String charsetName)
```

特有方法:

```
// 返回此流使用的字符编码的名称
String getEncoding()
```

//使用默认编码

```
InputStreamReader reader = new InputStreamReader(new  
FileInputStream("test.txt"));  
int len;  
while ((len = reader.read()) != -1) {  
    System.out.print((char) len);  
}  
reader.close();
```

//指定编码

```
InputStreamReader reader = new InputStreamReader(new  
FileInputStream("test.txt"), "utf-8");  
int len;  
while ((len = reader.read()) != -1) {  
    System.out.print((char) len);  
}  
reader.close();
```

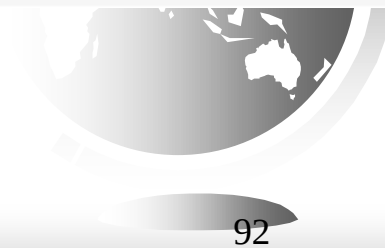
OutputStreamWriter: 字节流转字符流。

构造方法:

```
// 创建使用默认字符编码的 OutputStreamWriter
OutputStreamWriter(OutputStream out)
// 创建使用给定字符集的 OutputStreamWriter
OutputStreamWriter(OutputStream out, Charset cs)
// 创建使用给定字符集编码器的 OutputStreamWriter
OutputStreamWriter(OutputStream out, CharsetEncoder enc)
// 创建使用指定字符集的 OutputStreamWriter
OutputStreamWriter(OutputStream out, String charsetName)
```

特有方法:

```
// 返回此流使用的字符编码的名称
String getEncoding()
```



字符缓冲流

BufferedReader: 字符缓冲流，从字符输入流中读取文本，缓冲各个字符，从而实现字符、数组和行的高效读取。

构造方法:

// 创建一个使用默认大小输入缓冲区的缓冲字符输入流

BufferedReader(Reader in)

// 创建一个使用指定大小输入缓冲区的缓冲字符输入流

BufferedReader(Reader in, **int** sz)

特有方法:

// 读取一个文本行

String **readLine**()



//生成字符缓冲流对象

```
BufferedReader reader = new BufferedReader(new  
InputStreamReader(new FileInputStream("test.txt")));
```

```
String str;
```

//一次性读取一行

```
while ((str = reader.readLine()) != null) {  
    System.out.println(str);  
}
```

//关闭流

```
reader.close();
```



BufferedWriter: 字符缓冲流，将文本写入字符输出流，缓冲各个字符，从而提供单个字符、数组和字符串的高效写入。

构造方法:

```
// 创建一个使用默认大小输出缓冲区的缓冲字符输出流
```

```
BufferedWriter(Writer out)
```

```
// 创建一个使用给定大小输出缓冲区的新缓冲字符输出流
```

```
BufferedWriter(Writer out, int sz)
```

特有方法:

```
// 写入一个行分隔符
```

```
void newLine()
```



FileReader、FileWriter

FileReader: InputStreamReader类的直接子类，用来读取字符文件的便捷类，使用默认字符编码。

FileWriter: OutputStreamWriter类的直接子类，用来写入字符文件的便捷类，使用默认字符编码。



FileWriter

FileWriter是用来写字符流的，主要的构造函数有：

FileWriter(File file)

用File对象来构造FileWriter，写数据时，从文件开头开始写起，会覆盖以前的数据

FileWriter(File file, boolean append)

还是用File对象构造，如果第二个参数为true的话，表示以追加的方式写数据，从文件尾部开始写起

FileWriter(FileDescriptor fd)

用文件描述符来构造FileWriter

FileWriter(FileDescriptor fd, boolean append)

用文件描述符来构造，第二个参数为true的话，表示以追加的形式写入数据

FileWriter(String fileName)

用文件的路径名来构造FileWriter

FileWriter(String fileName, boolean append)

用文件路径名来构造FileWriter，第二个参数为true的话，表示以追加的形式写入文件



FileReader

用来方便的从文件中读出字符的类，默认编码和默认缓冲区大小假设是可以接受的。如果要改变默认编码和默认缓冲区大小可以用 `FileInputStream` 来构造 `InputStreamReader` 来实现。 `FileReader` 意味着是用来读字符的流，要实现读取字节流，请考虑使用 `FileInputStream`。
构造函数有：

`FileReader(File file)`

用 `File` 对象来构造 `FileReader`

`FileReader(FileDescriptor fd)`

用文件描述符构造 `FileReader`

`FileReader(String fileName)`

用文件的路径名来构造 `FileReader`

主要的函数大多集成于 `Reader` 类，请点击[这里](#)查看。其他主要的函数有：

`public String getEncoding()`

返回这个流使用的编码方式



```
public class FileWriter01 {  
    public static void main(String[] args) throws IOException {  
        FileWriter fw = new FileWriter("file01.txt");  
        fw.write(48);  
        fw.write(65);  
        fw.write(97);  
        fw.write(20013);  
        fw.close();  
    }  
}
```

显示出来的结果：0Aa中

ASCII 码表中常用的：48 - 0, 65 - A, 97 - a



write() 有五种重载方式:

1. **write(int args):** 写入数字, 利用 ASCII 码表 或者 Unicode 表转化
2. **write(String str):** 写入字符串
3. **write(String str,int offset,int count):** 写入字符串的一部分
4. **write(char[] array):** 写入字符数组
5. **write(char[] array,int offset,int count):** 写入字符数组的一部分

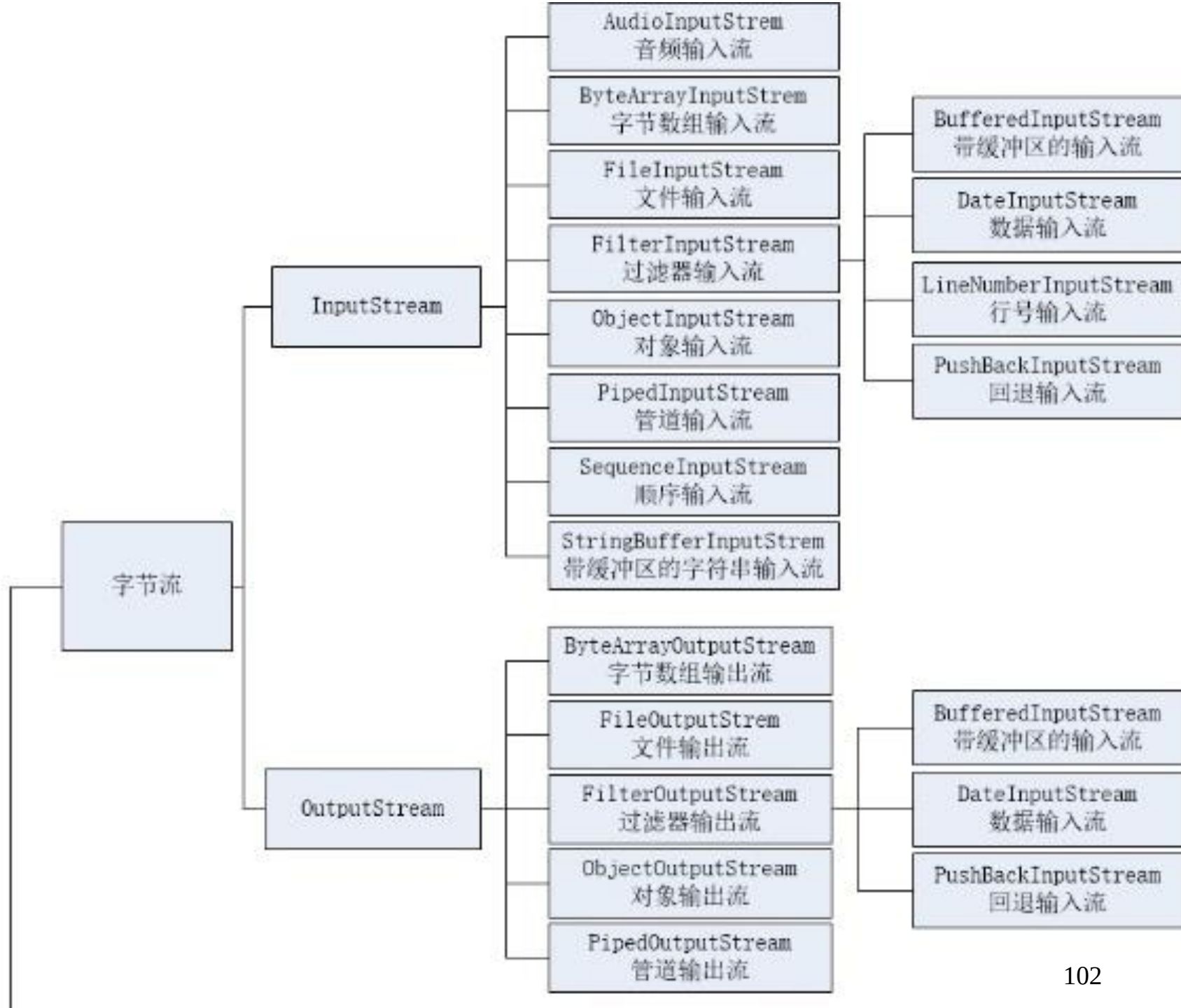


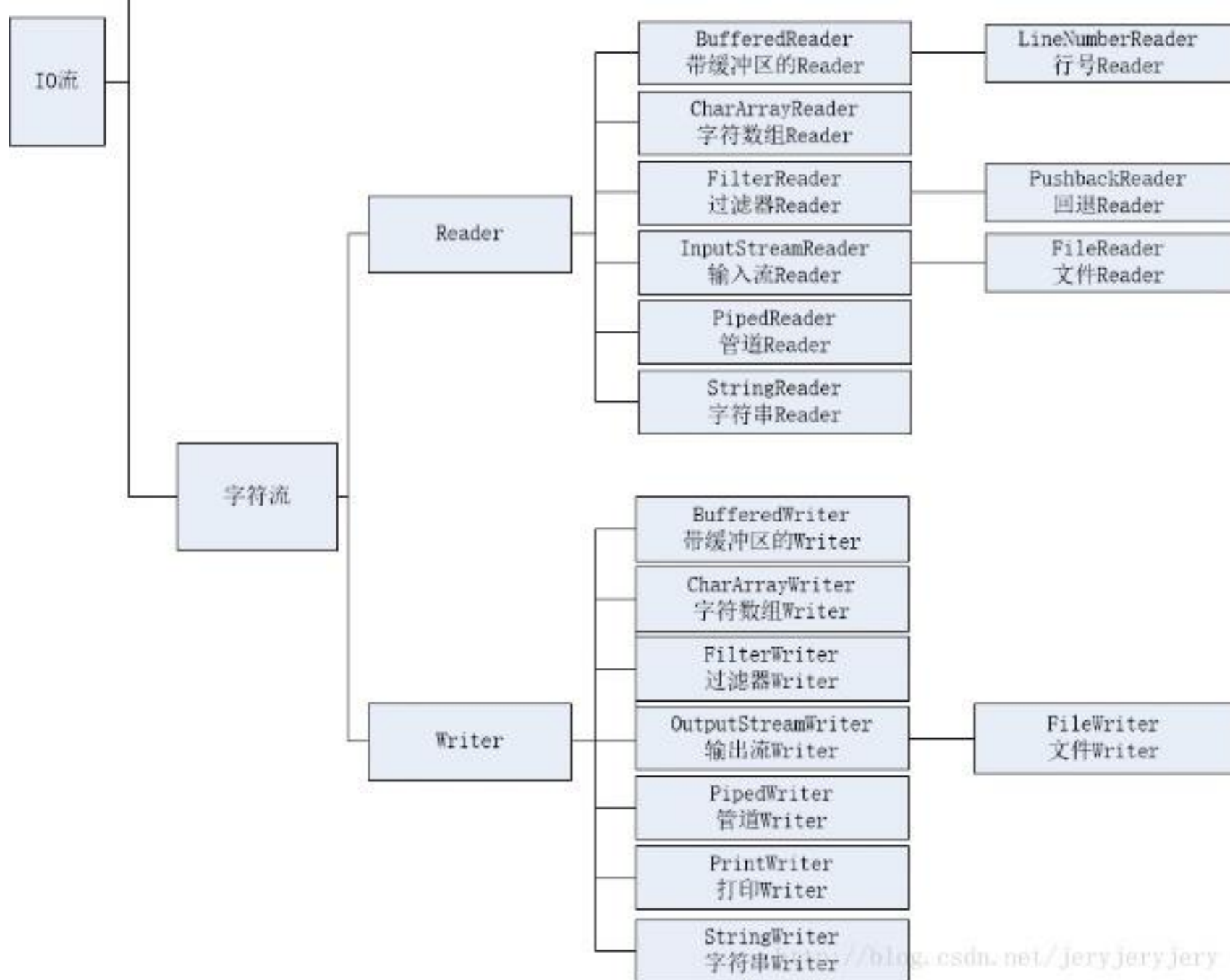
```
import java.io.*;

public class FileRead {
    public static void main(String args[]) throws IOException {
        File file = new File("Hello1.txt");
        // 创建文件
        file.createNewFile();
        // creates a FileWriter Object
        FileWriter writer = new FileWriter(file);
        // 向文件写入内容
        writer.write("This\n is\n an\n example\n");
        writer.flush();
        writer.close();
        // 创建 FileReader 对象
        FileReader fr = new FileReader(file);
        char[] a = new char[50];
        fr.read(a); // 从数组中读取内容
        for (char c : a)
            System.out.print(c); // 一个个打印字符
        fr.close();
    }
}
```

```
This
is
an
example
```







PipedOutputStream和PipedInputStream

PipedOutputStream和**PipedInputStream**分别是管道输出流和管道输入流。

它们的作用是**让多线程可以通过管道进行线程间的通讯**。在使用管道通信时，必须将PipedOutputStream和PipedInputStream配套使用。

使用管道通信时，大致的流程是：我们在线程A中向PipedOutputStream中写入数据，这些数据会自动的发送到与PipedOutputStream对应的PipedInputStream中，进而存储在PipedInputStream的缓冲中；此时，线程B通过读取PipedInputStream中的数据。就可以实现，线程A和线程B的通信。



PipedReader和PipedWriter

PipedReader和PipedWriter即管道输入流和输出流，可用于线程间管道通信。它们和PipedInputStream/PipedOutputStream区别是前者操作的是字符后者是字节。



PrintWriter和PrintStream

PrintStream主要操作byte流，而PrintWriter用来操作字符流。读取文本文件时一般用后者。

PrintStream和PrintWriter的API几乎相同，都能输出各种形式的数据，构造方法也几乎相同

- `PrintStream(File file)` - `java.io.PrintStream`
- `PrintStream(OutputStream out)` - `java.io.PrintStream`
- `PrintStream(String fileName)` - `java.io.PrintStream`
- `PrintStream(File file, String csn)` - `java.io.PrintStream`
- `PrintStream(OutputStream out, boolean autoFlush)` - `java.io.P`
- `PrintStream(String fileName, String csn)` - `java.io.PrintStream`
- `PrintStream(OutputStream out, boolean autoFlush, String enc)`
- `PrintStreamProviderFactory(PrintStream arg0)` - `sun.tracing.P`

PrintWriter和PrintStream

PrintWriter多了个接受Writer参数的构造函数

- `PrintWriter(File file)` - `java.io.PrintWriter`
- `PrintWriter(OutputStream out)` - `java.io.PrintWriter`
- `PrintWriter(String fileName)` - `java.io.PrintWriter`
- `PrintWriter(Writer out)` - `java.io.PrintWriter`
- `PrintWriter(File file, String csn)` - `java.io.PrintWriter`
- `PrintWriter(OutputStream out, boolean autoFlush)` - `java.io.Pr`
- `PrintWriter(String fileName, String csn)` - `java.io.PrintWriter`
- `PrintWriter(Writer out, boolean autoFlush)` - `java.io.PrintWite`

PrintWriter和PrintStream

```
new PrintStream("").p
try {
    Thread.sleep(300)
} catch (InterruptedException e) {
    // TODO Auto-generated method stub
    e.printStackTrace();
}

writer.println("write");
writer.close();
```

- print(boolean b) : void - PrintStream
- print(char c) : void - PrintStream
- print(char[] s) : void - PrintStream
- print(double d) : void - PrintStream
- print(float f) : void - PrintStream
- print(int i) : void - PrintStream
- print(long l) : void - PrintStream
- print(Object obj) : void - PrintStream
- print(String s) : void - PrintStream
- printf(String format, Object[] args) : PrintStream - PrintStream
- printf(Locale l, String format, Object[] args) : PrintStream - PrintStream
- println() : void - PrintStream
- println(boolean x) : void - PrintStream
- println(char x) : void - PrintStream
- println(char[] x) : void - PrintStream
- println(double x) : void - PrintStream

```
new PrintWriter("").p
try {
    Thread.sleep(300)
} catch (InterruptedException e) {
    // TODO Auto-generated method stub
    e.printStackTrace();
}

writer.println("write");
writer.close();
```

- print(boolean b) : void - PrintWriter
- print(char c) : void - PrintWriter
- print(char[] s) : void - PrintWriter
- print(double d) : void - PrintWriter
- print(float f) : void - PrintWriter
- print(int i) : void - PrintWriter
- print(long l) : void - PrintWriter
- print(Object obj) : void - PrintWriter
- print(String s) : void - PrintWriter
- printf(String format, Object[] args) : PrintWriter - PrintWriter
- printf(Locale l, String format, Object[] args) : PrintWriter - PrintWriter
- println() : void - PrintWriter
- println(boolean x) : void - PrintWriter
- println(char x) : void - PrintWriter
- println(char[] x) : void - PrintWriter
- println(double x) : void - PrintWriter

PrintWriter和PrintStream

两个类的功能基本相同，**PrintStream**能做的**PrintWriter**也都能实现，并且**PrintWriter**的功能更为强大。但是由于PrintWriter出现的比较晚，较早的System.out使用的是PrintStream来实现的，所以为了兼容就没有废弃PrintStream。

两个类最大的差别是，PrintStream在输出字符，将字符转换为字节时采用的是**系统默认的编码格式**，这样当数据传输另一个平台，而另一个平台使用另外一个编码格式解码时就会出现问题，存在不可控因素。而PrintWriter可以在传入Writer时，**可由程序员指定字符转换为字节时的编码格式**，这样兼容性和可控性会更好。

